

Chapter 11

Final Project

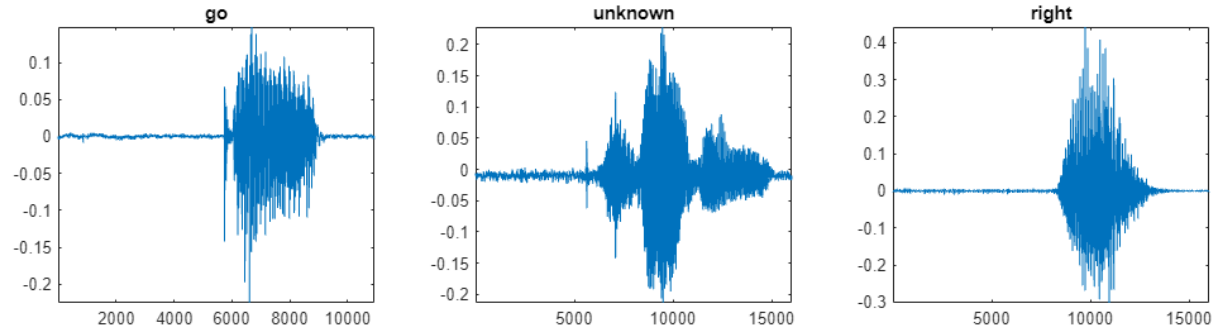


MLP Model

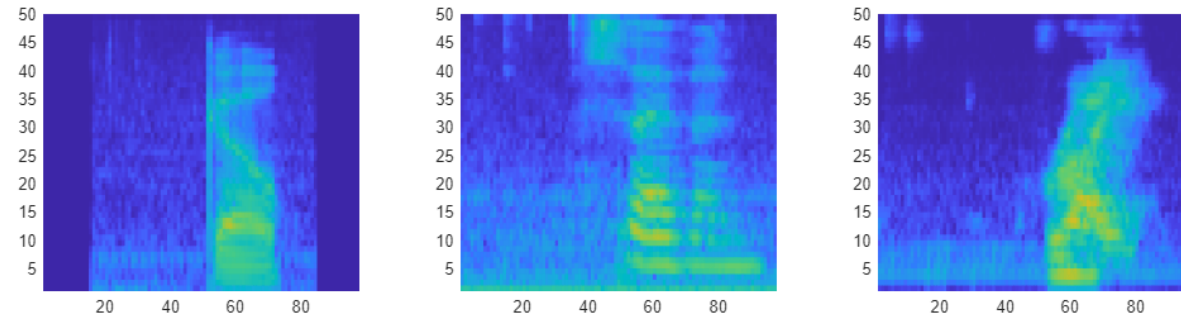


MLP Model – dataset

Time domain

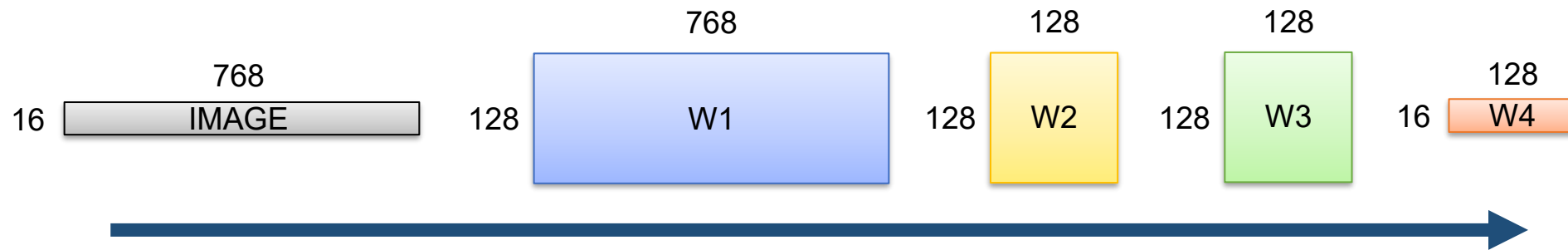


Freq. domain



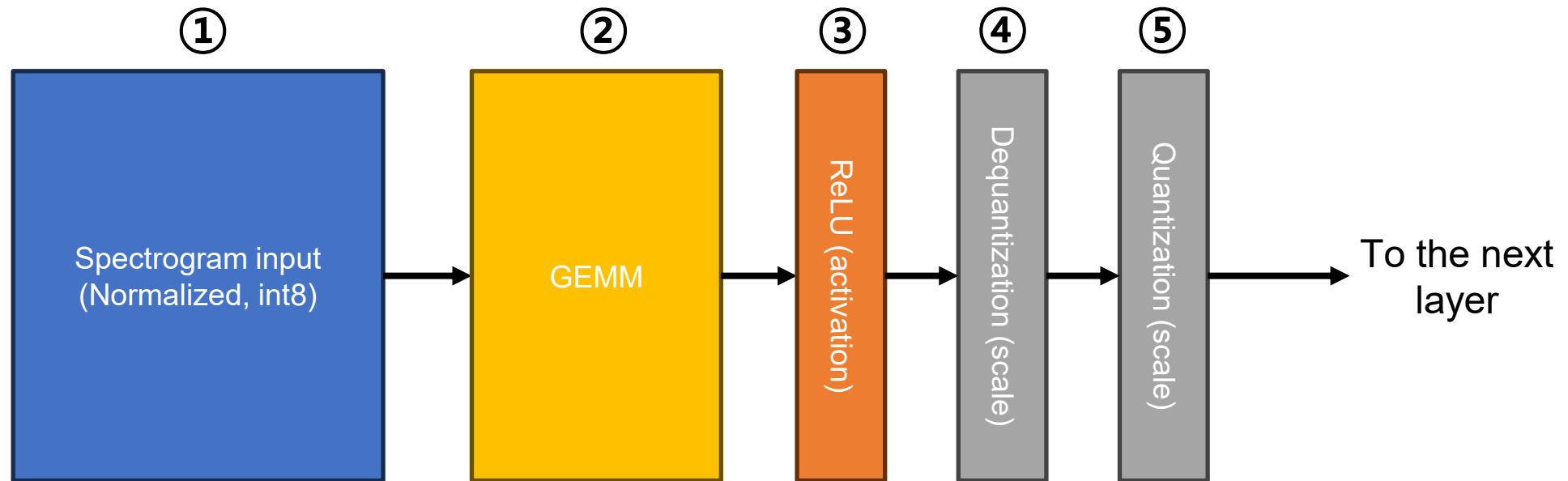
- Google speech command dataset : 1-2초 내외의 짧은 길이로 이루어진 음성 명령 데이터셋
- 음성 데이터를 FFT하여 spectrogram으로 변환한 이미지를 사용할 것임
- 각 음성 명령의 spectrogram상 특징을 사용하여 0-9 숫자를 classification하는 것이 목표

MLP Model – architecture



- **4-layer MLP모델 활용**
- Systolic array는 GEMV가 아닌 GEMM연산에 유리함 -> **flatten한 1×768 spectrogram 이미지를 16개 batch**
- Spectrogram이미지는 이미 normalization을 수행함
- 4개 레이어, 각 행렬들이 16×16 시스톨릭 어레이에 맞지 않으므로 **타일링 필요**
- **난이도 조절을 위해 미리 타일링을 해두었습니다**

MLP Model – Process pipeline



- ① - ⑤를 4번 반복 (4-layer)
- ④와 ⑤의 경우, dequant/quant scaling factor를 2개 합친 1개의 factor를 제공
- 전 과정에 걸쳐 부호 있는 정수 연산만 필요



Recap: 2D-Systolic Array

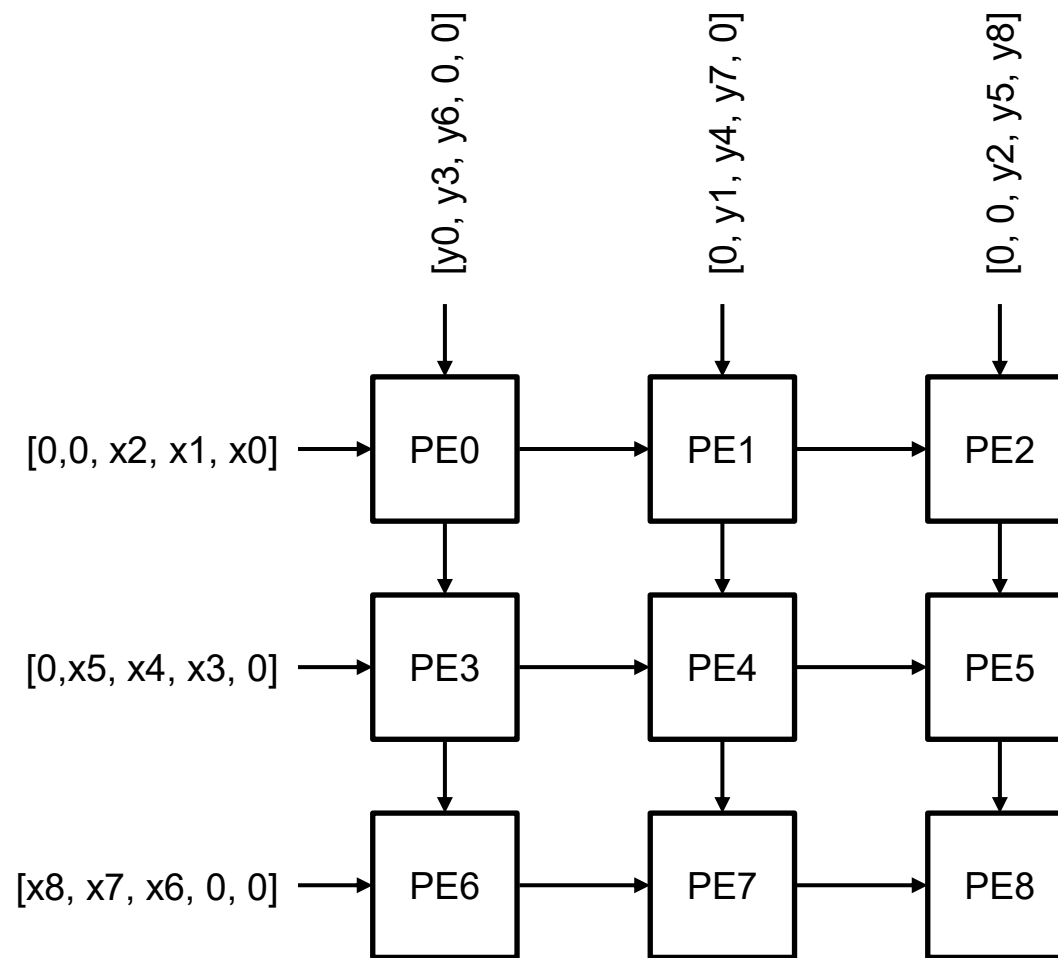


2D-Systolic Array

$$A = \begin{bmatrix} x_0 & x_1 & x_2 \\ x_3 & x_4 & x_5 \\ x_6 & x_7 & x_8 \end{bmatrix}$$

$$B = \begin{bmatrix} y_0 & y_1 & y_2 \\ y_3 & y_4 & y_5 \\ y_6 & y_7 & y_8 \end{bmatrix}$$

$$C = \begin{bmatrix} x_0 * y_0 & x_0 * y_1 & x_0 * y_2 \\ +x_1 * y_3 & +x_1 * y_4 & +x_1 * y_5 \\ +x_2 * y_6 & +x_2 * y_7 & +x_2 * y_8 \\ x_3 * y_0 & x_3 * y_1 & x_3 * y_2 \\ +x_4 * y_3 & +x_4 * y_4 & +x_4 * y_5 \\ +x_5 * y_6 & +x_5 * y_7 & +x_5 * y_8 \\ x_6 * y_0 & x_6 * y_1 & x_6 * y_2 \\ +x_7 * y_3 & +x_7 * y_4 & +x_7 * y_5 \\ +x_8 * y_6 & +x_8 * y_7 & +x_8 * y_8 \end{bmatrix}$$

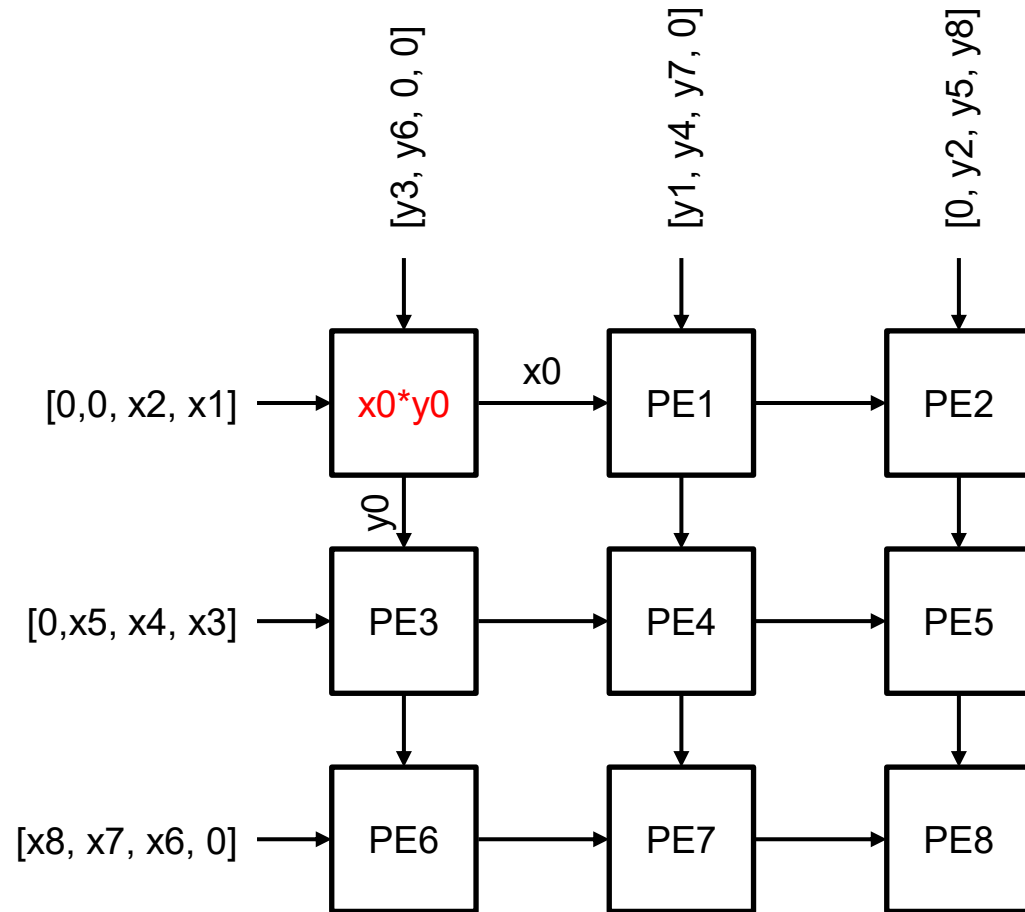


2D-Systolic Array

$$A = \begin{bmatrix} x0 & x1 & x2 \\ x3 & x4 & x5 \\ x6 & x7 & x8 \end{bmatrix}$$

$$B = \begin{bmatrix} y0 & y1 & y2 \\ y3 & y4 & y5 \\ y6 & y7 & y8 \end{bmatrix}$$

$$C = \begin{bmatrix} x0 * y0 & x0 * y1 & x0 * y2 \\ +x1 * y3 & +x1 * y4 & +x1 * y5 \\ +x2 * y6 & +x2 * y7 & +x2 * y8 \\ x3 * y0 & x3 * y1 & x3 * y2 \\ +x4 * y3 & +x4 * y4 & +x4 * y5 \\ +x5 * y6 & +x5 * y7 & +x5 * y8 \\ x6 * y0 & x6 * y1 & x6 * y2 \\ +x7 * y3 & +x7 * y4 & +x7 * y5 \\ +x8 * y6 & +x8 * y7 & +x8 * y8 \end{bmatrix}$$

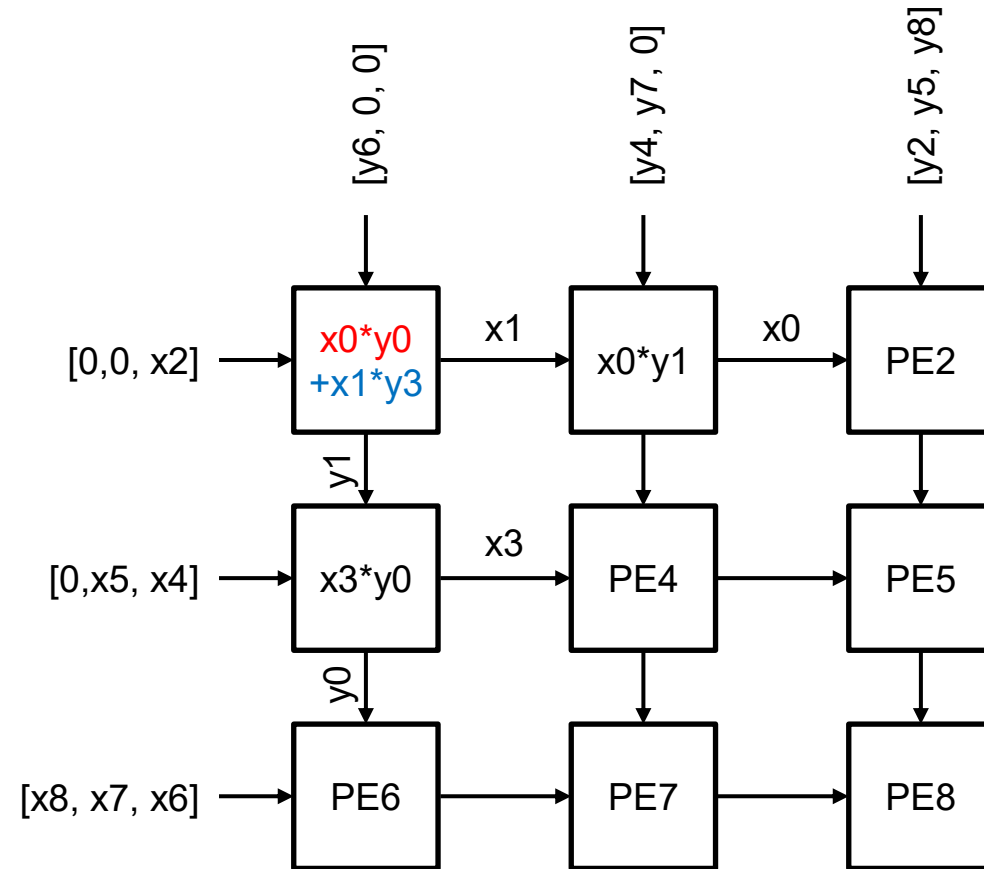


2D-Systolic Array

$$A = \begin{bmatrix} x_0 & x_1 & x_2 \\ x_3 & x_4 & x_5 \\ x_6 & x_7 & x_8 \end{bmatrix}$$

$$B = \begin{bmatrix} y_0 & y_1 & y_2 \\ y_3 & y_4 & y_5 \\ y_6 & y_7 & y_8 \end{bmatrix}$$

$$C = \begin{bmatrix} x_0 * y_0 & x_0 * y_1 & x_0 * y_2 \\ +x_1 * y_3 & +x_1 * y_4 & +x_1 * y_5 \\ +x_2 * y_6 & +x_2 * y_7 & +x_2 * y_8 \\ x_3 * y_0 & x_3 * y_1 & x_3 * y_2 \\ +x_4 * y_3 & +x_4 * y_4 & +x_4 * y_5 \\ +x_5 * y_6 & +x_5 * y_7 & +x_5 * y_8 \\ x_6 * y_0 & x_6 * y_1 & x_6 * y_2 \\ +x_7 * y_3 & +x_7 * y_4 & +x_7 * y_5 \\ +x_8 * y_6 & +x_8 * y_7 & +x_8 * y_8 \end{bmatrix}$$

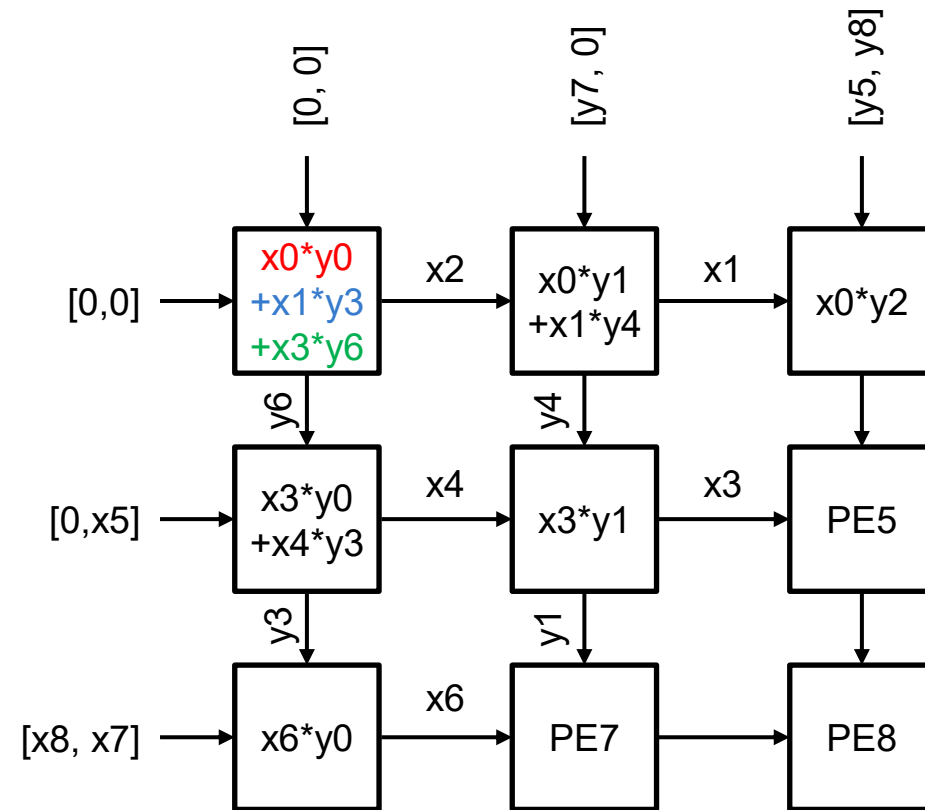


2D-Systolic Array

$$A = \begin{bmatrix} x0 & x1 & x2 \\ x3 & x4 & x5 \\ x6 & x7 & x8 \end{bmatrix}$$

$$B = \begin{bmatrix} y0 & y1 & y2 \\ y3 & y4 & y5 \\ y6 & y7 & y8 \end{bmatrix}$$

$$C = \begin{bmatrix} x0 * y0 & x0 * y1 & x0 * y2 \\ +x1 * y3 & +x1 * y4 & +x1 * y5 \\ +x2 * y6 & +x2 * y7 & +x2 * y8 \\ x3 * y0 & x3 * y1 & x3 * y2 \\ +x4 * y3 & +x4 * y4 & +x4 * y5 \\ +x5 * y6 & +x5 * y7 & +x5 * y8 \\ x6 * y0 & x6 * y1 & x6 * y2 \\ +x7 * y3 & +x7 * y4 & +x7 * y5 \\ +x8 * y6 & +x8 * y7 & +x8 * y8 \end{bmatrix}$$

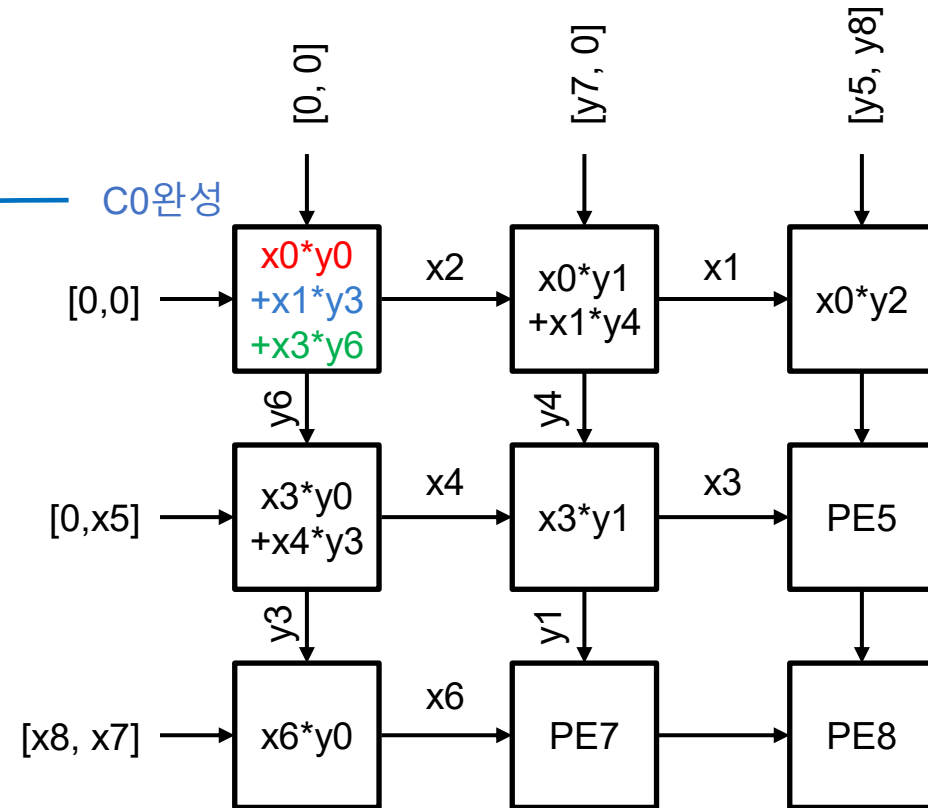


2D-Systolic Array

$$A = \begin{bmatrix} x_0 & x_1 & x_2 \\ x_3 & x_4 & x_5 \\ x_6 & x_7 & x_8 \end{bmatrix}$$

$$B = \begin{bmatrix} y_0 & y_1 & y_2 \\ y_3 & y_4 & y_5 \\ y_6 & y_7 & y_8 \end{bmatrix}$$

$$C = \begin{bmatrix} x_0 * y_0 & x_0 * y_1 & x_0 * y_2 \\ +x_1 * y_3 & +x_1 * y_4 & +x_1 * y_5 \\ +x_2 * y_6 & +x_2 * y_7 & +x_2 * y_8 \\ x_3 * y_0 & x_3 * y_1 & x_3 * y_2 \\ +x_4 * y_3 & +x_4 * y_4 & +x_4 * y_5 \\ +x_5 * y_6 & +x_5 * y_7 & +x_5 * y_8 \\ x_6 * y_0 & x_6 * y_1 & x_6 * y_2 \\ +x_7 * y_3 & +x_7 * y_4 & +x_7 * y_5 \\ +x_8 * y_6 & +x_8 * y_7 & +x_8 * y_8 \end{bmatrix}$$

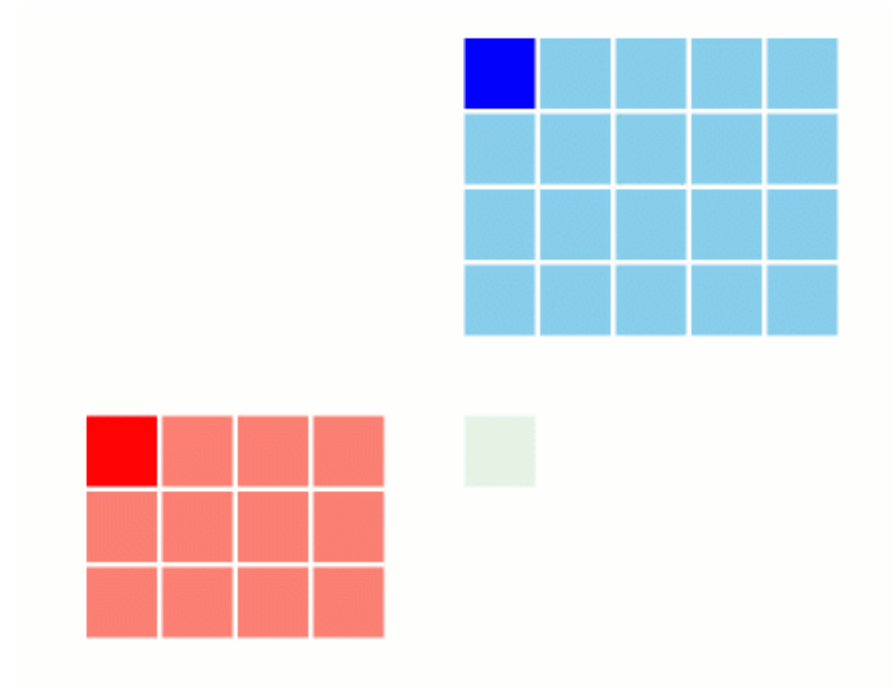




Recap: Tiling

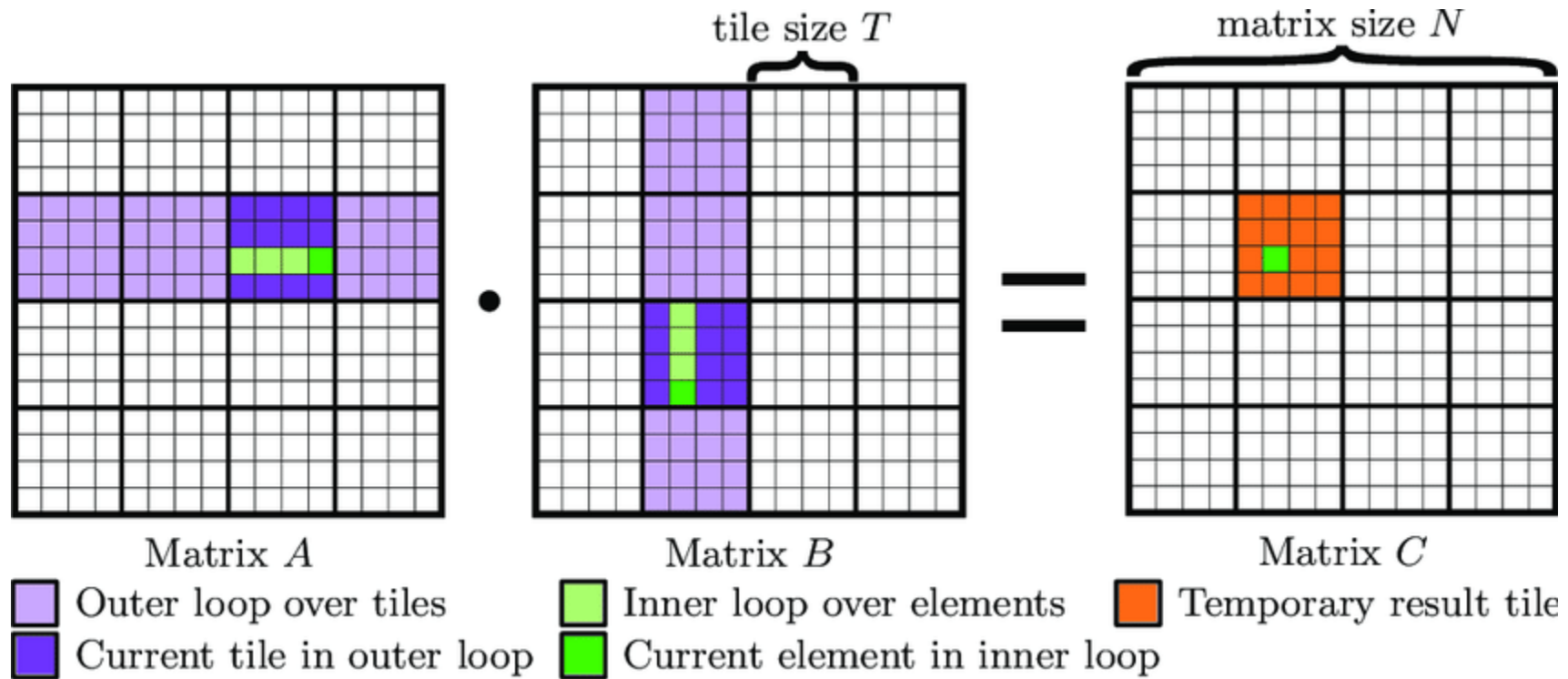


Tiling



- 타일링
 - 1개의 시스톨릭 어레이보다 큰 행렬을 부분행렬로 나누어 partial sum을 accumulate하는 방식
 - 메모리상의 큰 행렬은 타일 단위로 쪼개져있지 않음 -> 타일 원소의 실제 어드레스 계산이 필요

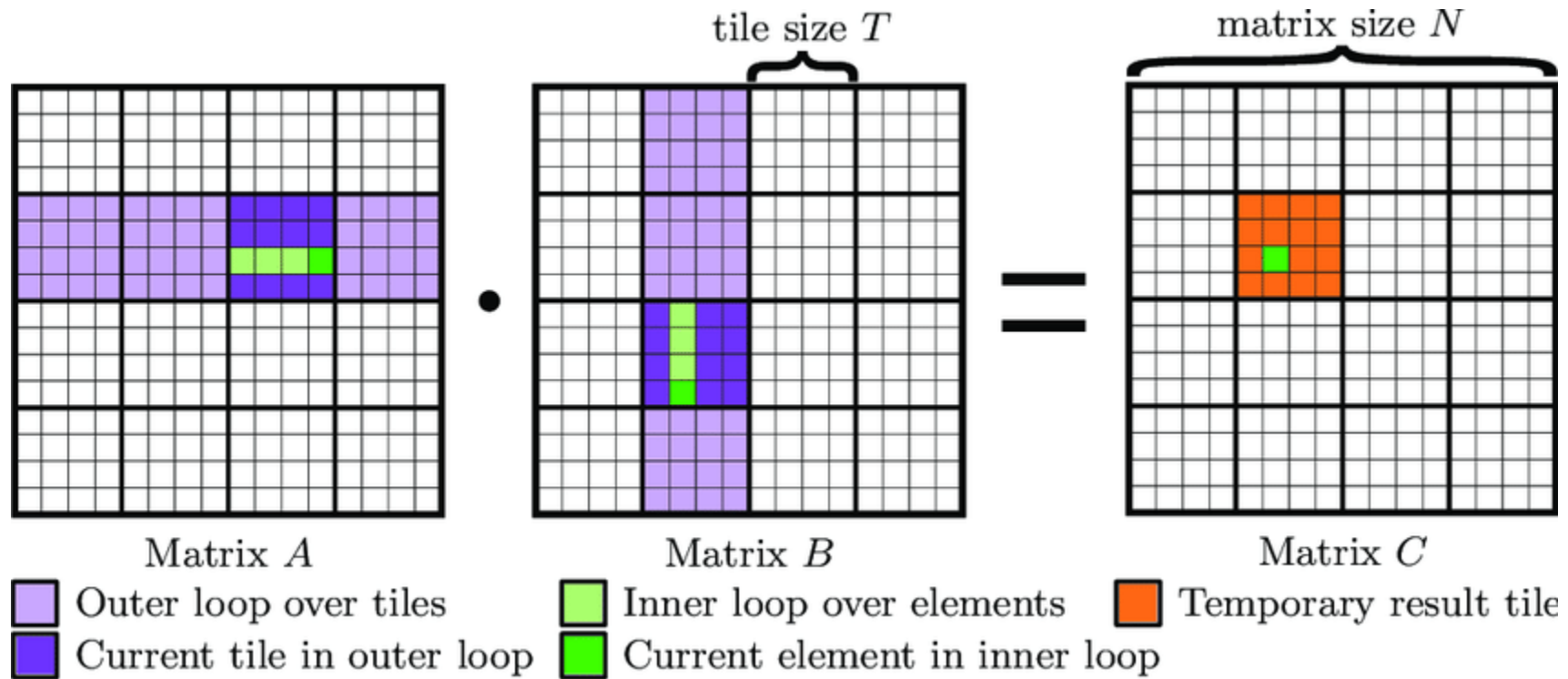
Tiling



- 타일링

- 1개의 시스톨릭 어레이보다 큰 행렬을 부분행렬로 나누어 partial sum을 누산하는 방식
- 메모리상의 큰 행렬은 타일 단위로 쪼개져있지 않음 -> 타일 원소의 실제 어드레스 계산이 필요

Tiling



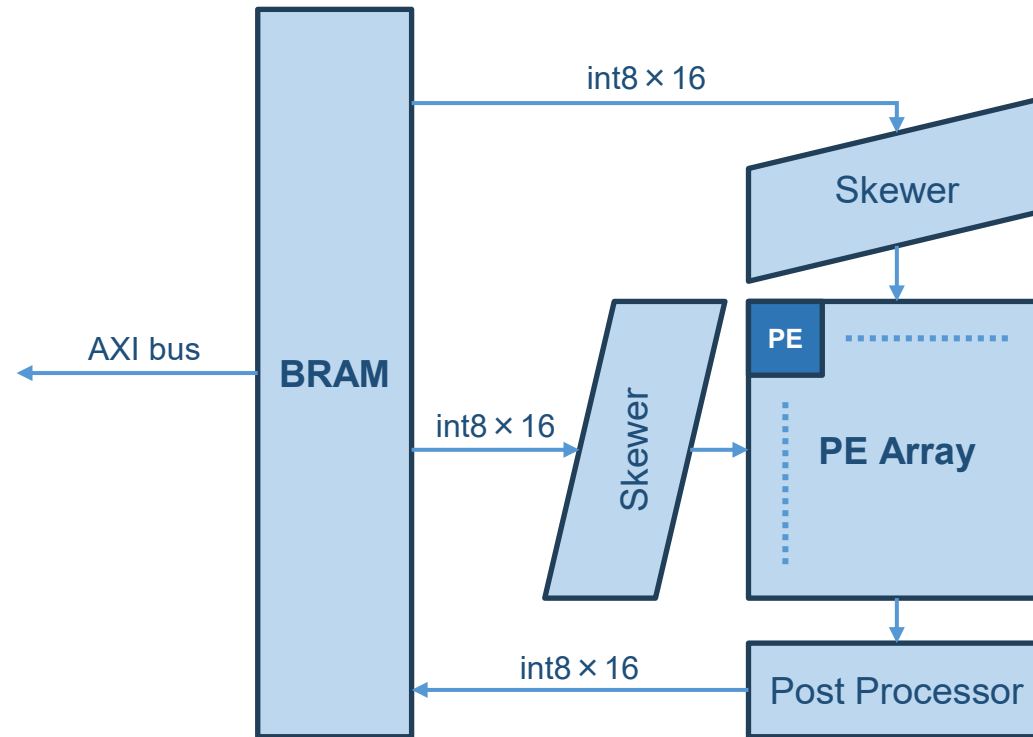
- 타일링

- 1개의 시스톨릭 어레이보다 큰 행렬을 부분행렬로 나누어 partial sum을 누산하는 방식
- 메모리상의 큰 행렬은 타일 단위로 쪼개져있지 않음 -> 타일 원소의 실제 어드레스 계산이 필요
- 타일링을 미리 해 둔 데이터를 제공 (실제 어드레스 계산 모듈을 만들 필요 없음)



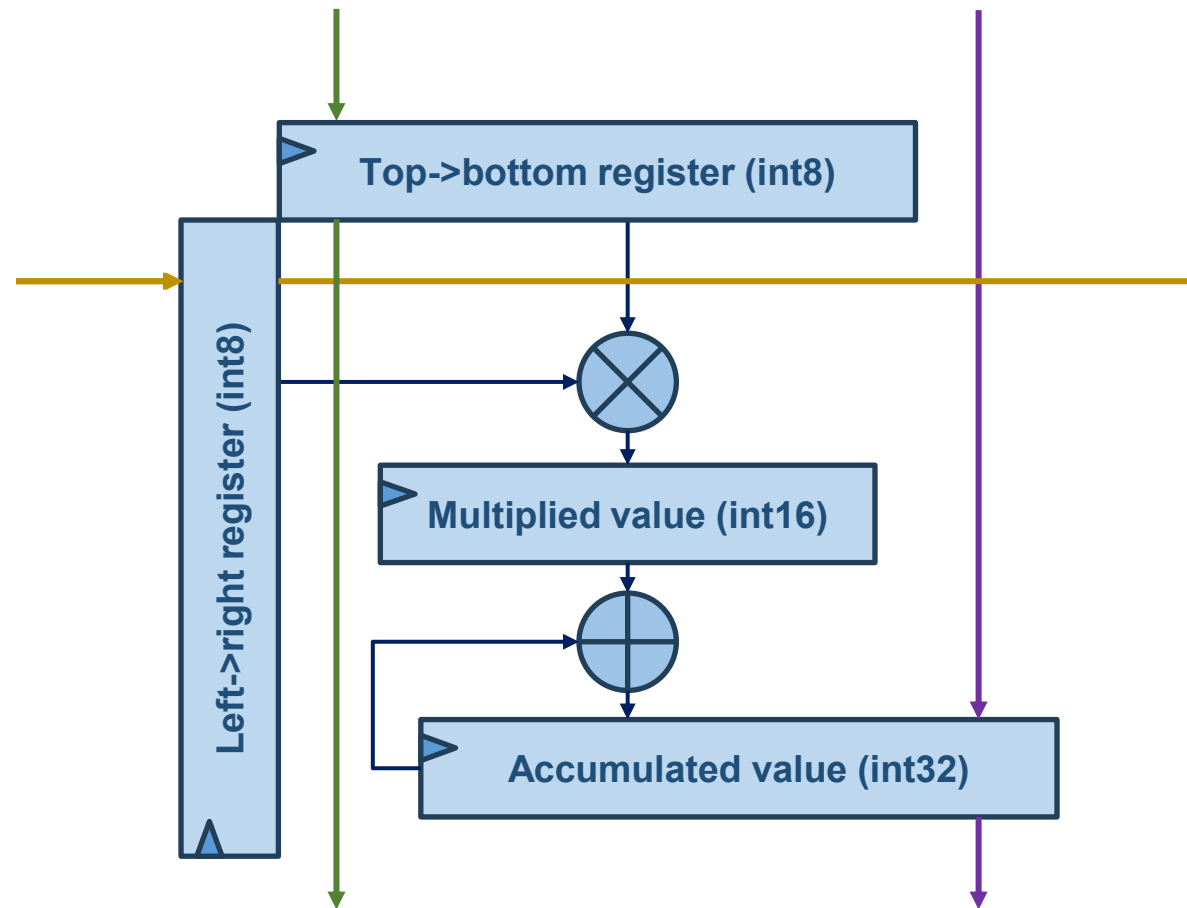
Structure

Structure



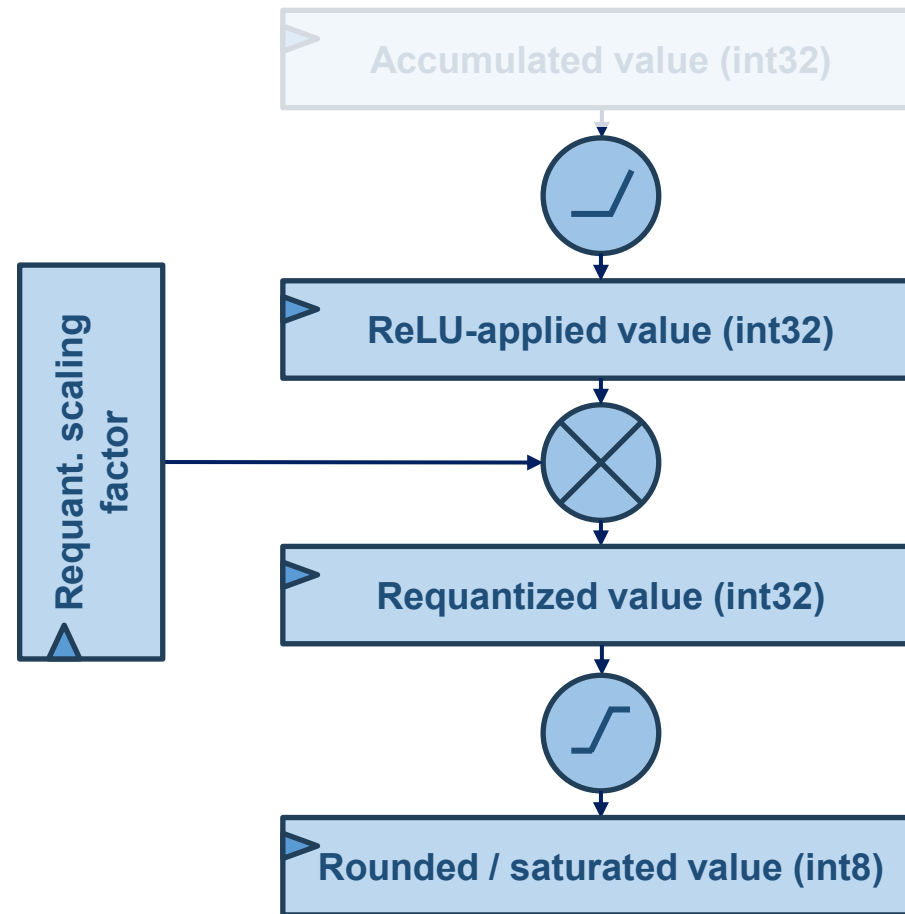
- 시스톨릭 어레이
 - 스트리밍 데이터용 BRAM+시스톨릭 어레이 구조
 - 여러분이 설계/구현할 것은 BRAM을 제외한 전부

Structure – PE



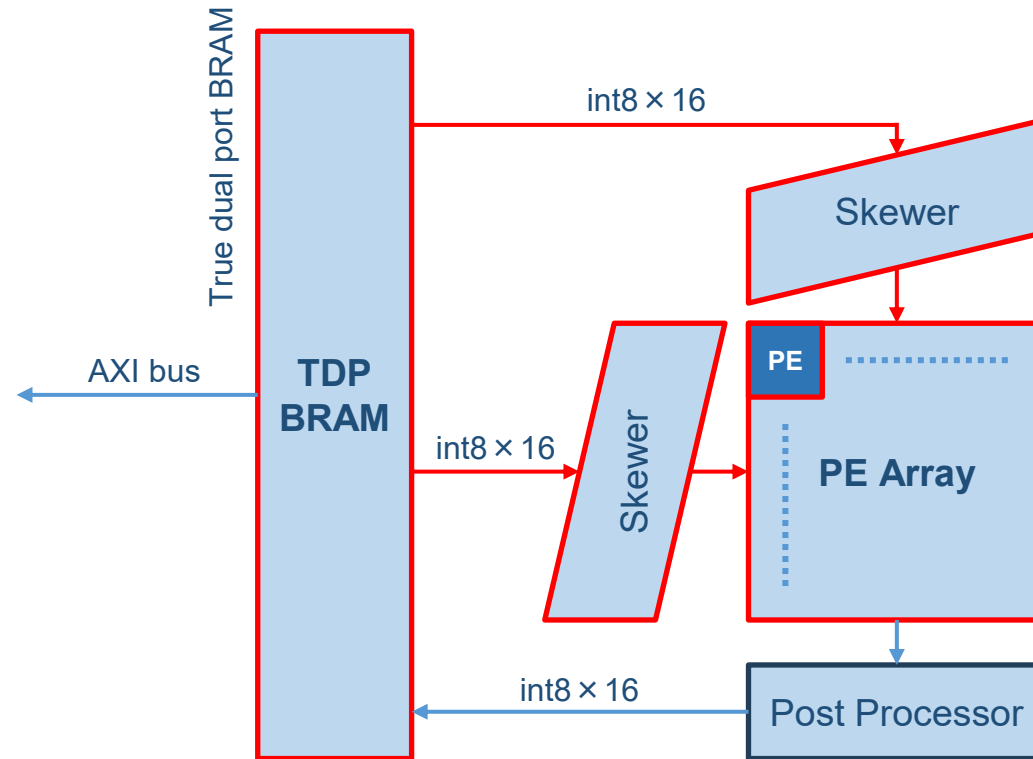
- PE
 - 시프트 레지스터 + multiplier + accumulator
 - Processing state에서는 int8을 오른쪽/아랫쪽으로 시프트, draining state에서는 int32를 아래로 시프트

Structure – Post Processor



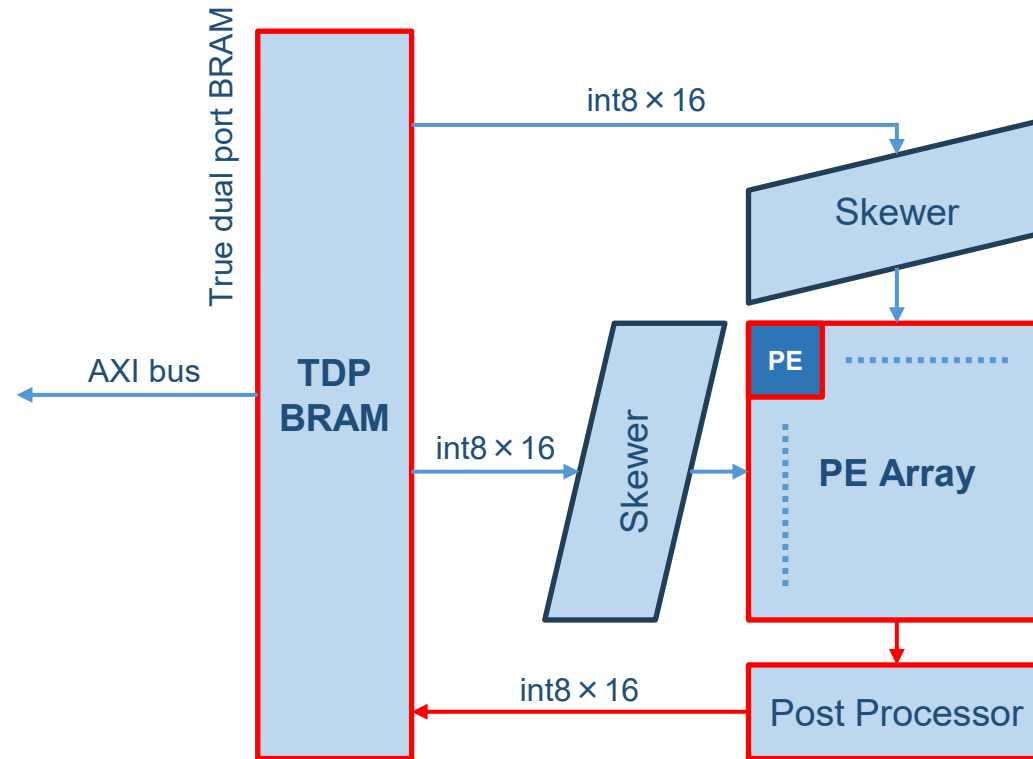
- Post processor
 - Systolic array의 32-bit accumulated value를 1-row씩 draining할때 작동
 - ReLU -> requantization -> rounding/saturating 순서의 파이프라인

Structure – Sequencer



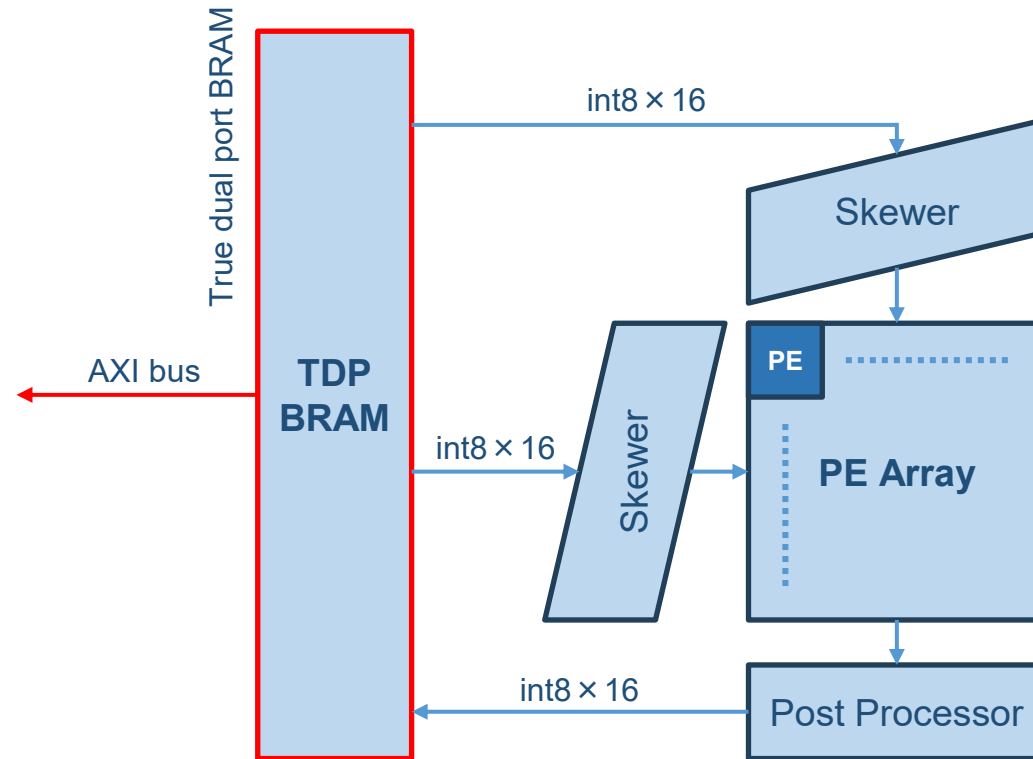
- 시퀀싱 순서
 1. BRAM에서 2개의 128바이트 ($\text{int8} \times 16$) 값을 인출하여 PE array에 multiply-accumulate

Structure – Sequencer



- 시퀀싱 순서
 1. BRAM에서 2개의 128바이트 ($\text{int8} \times 16$) 값을 인출하여 PE array에 multiply-accumulate
 2. PE array에서 accumulation이 모두 끝난 값들을 post-processing 후 BRAM에 기록

Structure – Sequencer



- 시퀀싱 순서

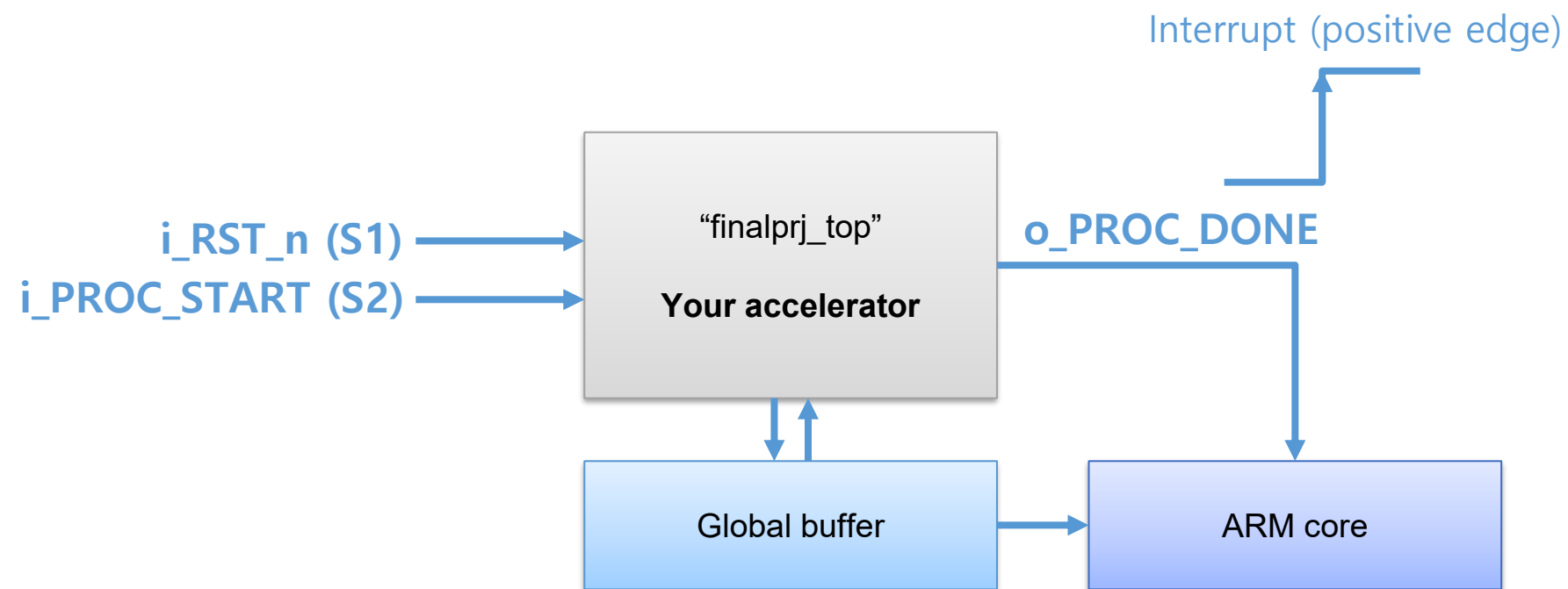
1. BRAM에서 2개의 128바이트 ($\text{int8} \times 16$) 값을 인출하여 PE array에 multiply-accumulate
2. PE array에서 accumulation이 모두 끝난 값들을 post-processing 후 BRAM에 기록
3. 모든 레이어의 연산이 끝났다면, ARM이 BRAM의 값을 읽어 확인 (인터럽트 사용)

Structure – Sequencer

- 시퀀싱 시 고려점
 - 3중 for문을 하드웨어로 제어
 - Sequencer (finite state machine) 을 “함수화” 해서 얻는 이점 극대화 필요
 - 파이프라인의 적절한 제어 필수
- 어떤 작업을 함수화 할까? → 보고서에 필수 작성
- 무엇을 argument로 넘길까?

```
start process
  for layer      = 0 ... 3
  | for tile_row = 0 ... N
  | | for tile_col = 0 ... M
  | | | → stream data (read from BRAM)
  | | | → accumulate partial sums
  | | → drain the PE array (write to BRAM)
  | → to the next layer
assert IRQ
```

Structure – I/O





Material description

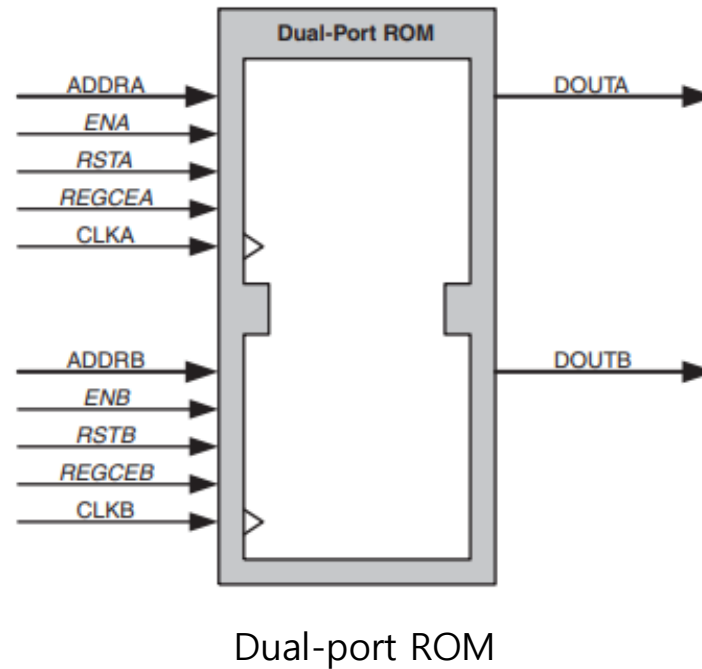


Material description

- Python reference
 - numpy_reference.py - 추론 과정 레퍼런스 파일 (Google Colab 또는 local에서 실행)
 - weights/*.bin - 레이어별 가중치
 - weights/bin2hex.py - 원본 가중치 -> readmemh용 ASCII 포맷 txt변환기 (with 사전 타일링)
- Vivado project
 - Finalprj_srcs/finalprj_wrapper.v - wrapper파일
 - Finalprj_srcs/finalprj_top.sv - top파일 (Verilog/SystemVerilog 모두 사용 가능, 포트 바꾸지 말기)
 - Finalprj_srcs/bram_init.txt - BRAM 초기화 파일 (변경 불가)
 - helloworld.c - Vitis프로젝트 C소스 복사본
- USB-Serial adapter driver
 - USB-to-Serial_PL2303_Prolific_DriverInstaller_v1.12.0 - 컴퓨터에서 UART어댑터 지원 안 할 경우 사용

Material description – TDP BRAM

- ▶ True dual port BRAM을 사용합니다. 2개의 포트 각각 다른 어드레스를 줄 수 있습니다.



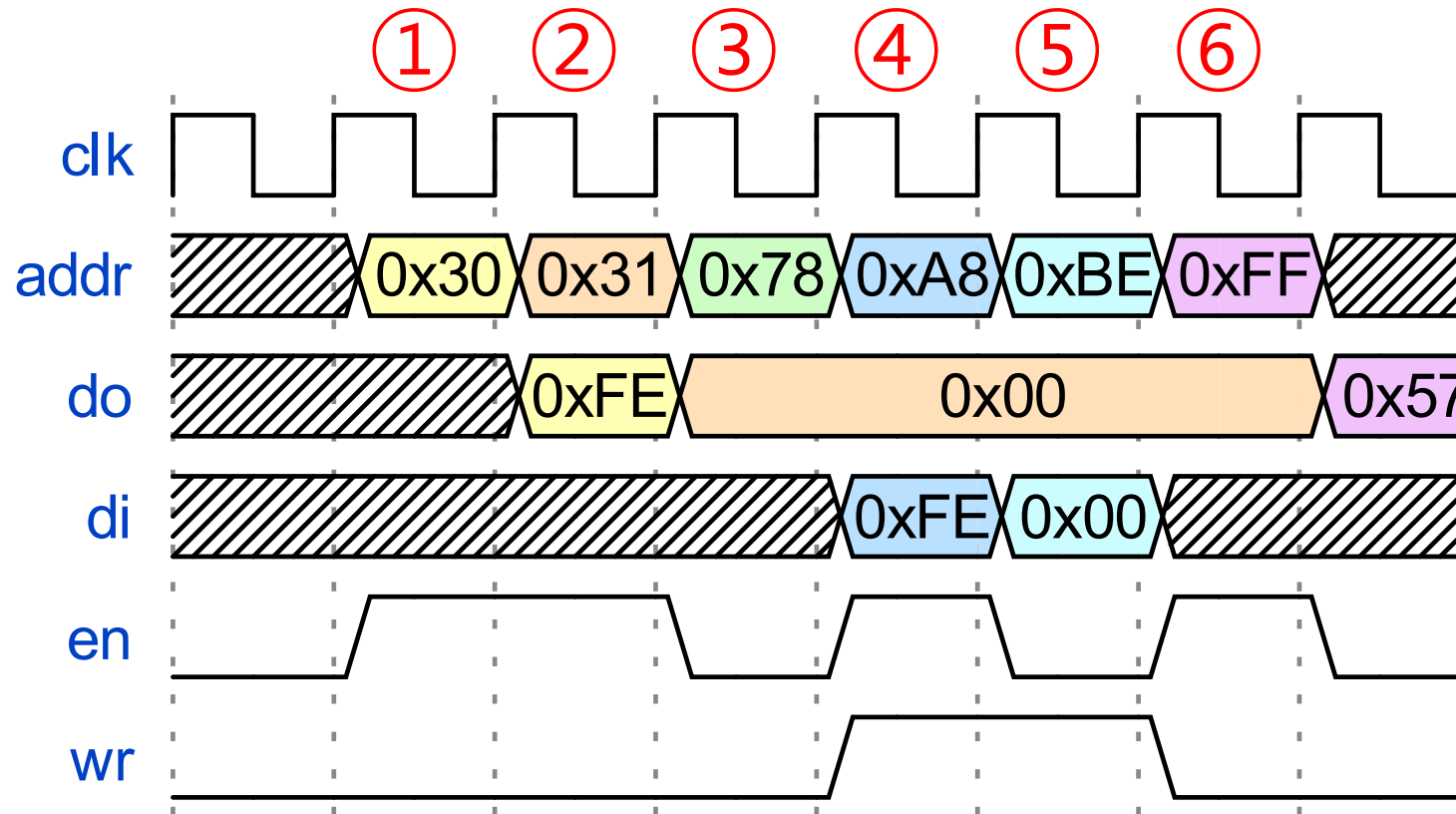
Material description – TDP BRAM

```
BRAM_TDP #(
    .INIT_FILE          ("bram_init.txt"           ) //초기화 파일
) u_bram (
    //Port A – I/O path (read input matrix, write output matrix) + AXI
    .i_PA_ADDR          (14'd0                     ), //포트 A 어드레스
    .i_PA_WR            (1'b0                       ), //포트 A 쓰기 스트로브
    .i_PA_WDATA         (128'd0                    ), //포트 A 쓰기 데이터
    .o_PA_RDATA         (                           ), //포트 A 읽기 데이터
    .o_PA_BUSY          (                           ), //미사용

    //Port B – weight path (read only from RTL side)
    .i_PB_ADDR          (14'd0                     ), //포트 B 어드레스
    .i_PB_WR            (1'b0                       ), //포트 B 쓰기 스트로브
    .i_PB_WDATA         (128'd0                    ), //포트 B 쓰기 데이터
    .o_PB_RDATA         (                           ), //포트 B 읽기 데이터
```

Material description – TDP BRAM

► BRAM(one port) R/W timing diagram



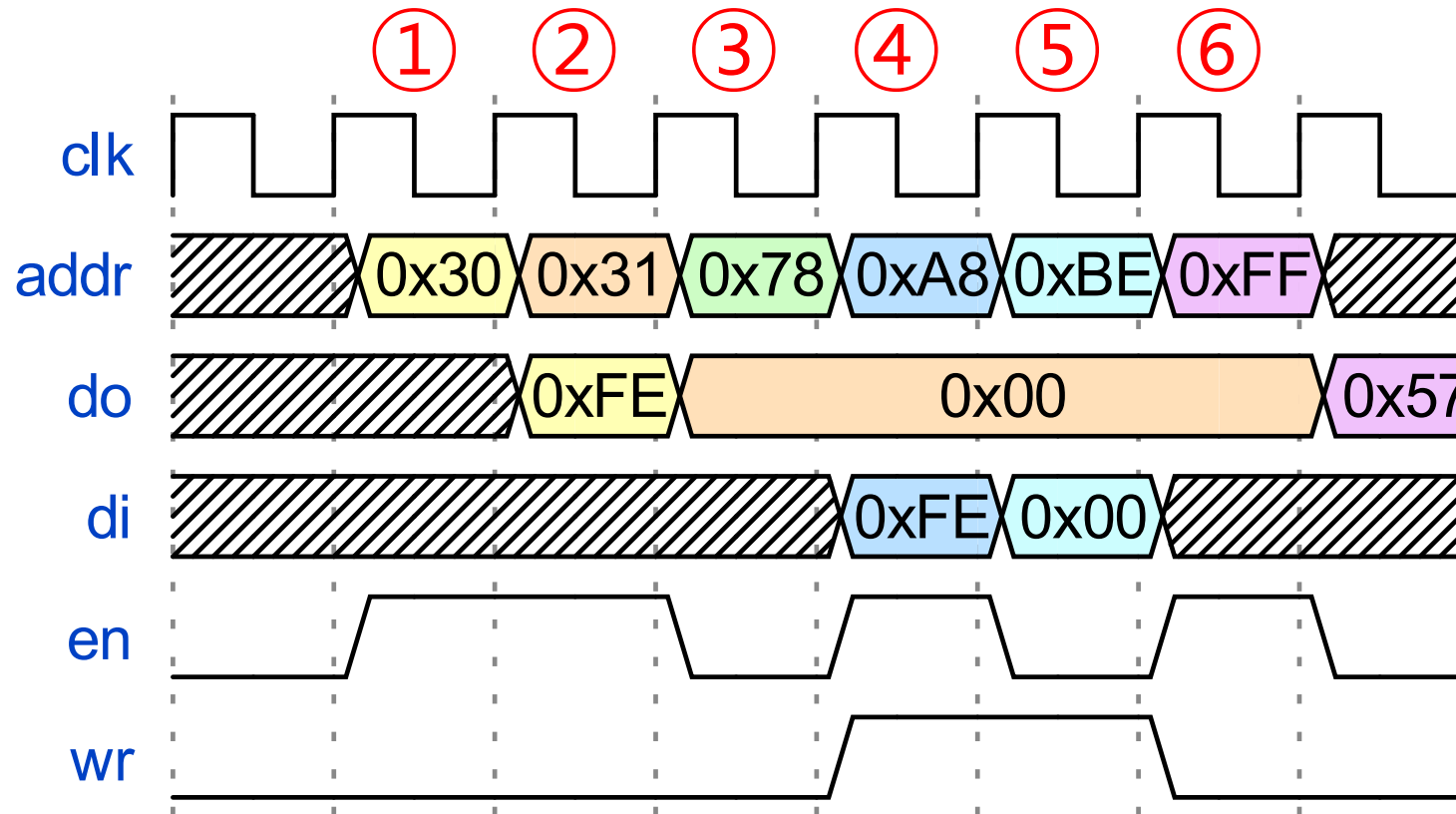
Cycle 1: read address 0x30 (en = 1)
data appears on "do" at the next cycle
read data = 0xFE

Cycle 2: read address 0x31 (en = 1)
data appears on "do" at the next cycle
read data = 0x00

Cycle 3: no operation 0x78 (en = 0)
"do" retains the previous value

Material description – TDP BRAM

► BRAM(one port) R/W timing diagram



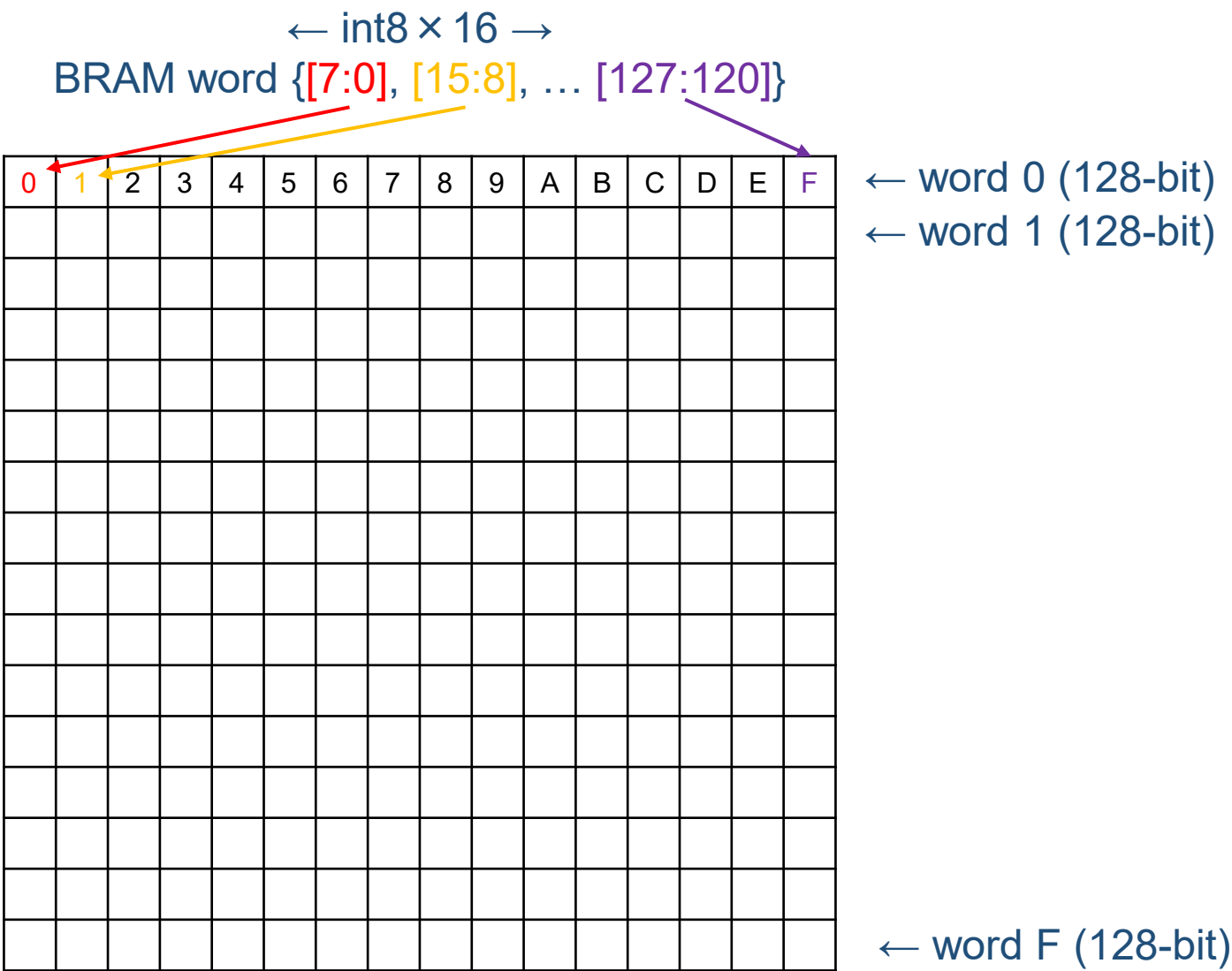
Cycle 4: write address 0xA8 (en = 1)
"do" retains the previous value
write data = 0xFE

Cycle 5: no operation 0xBE (en = 0)
"do" retains the previous value

Cycle 6: read address 0xFF (en = 1)
data appears on "do" at the next cycle
read data = 0x57

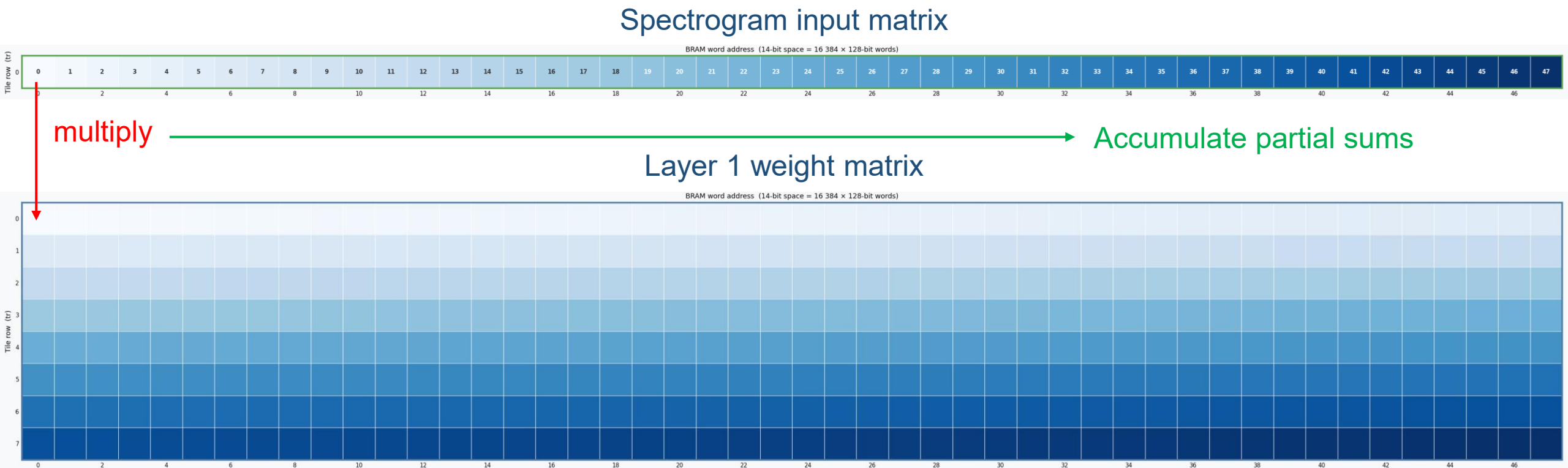
Material description – data structure

- Single tile
 - Row-wise로 저장됨



Material description – data structure

- Matrices
 - Tile-wise로 저장됨 (각각의 블록은 1개의 타일을 나타냄)



Material description – data structure

- Matrices
 - Tile-wise로 저장됨 (각각의 블록은 1개의 타일을 나타냄)

Layer 2 weight matrix

BRAM word address (14-bit space = 16 384 × 128-bit words)

0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63
	0	1	2	3	4	5	6	7

Layer 3 weight matrix

BRAM word address (14-bit space = 16 384 × 128-bit words)

0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63
	0	1	2	3	4	5	6	7

Layer 4 weight matrix

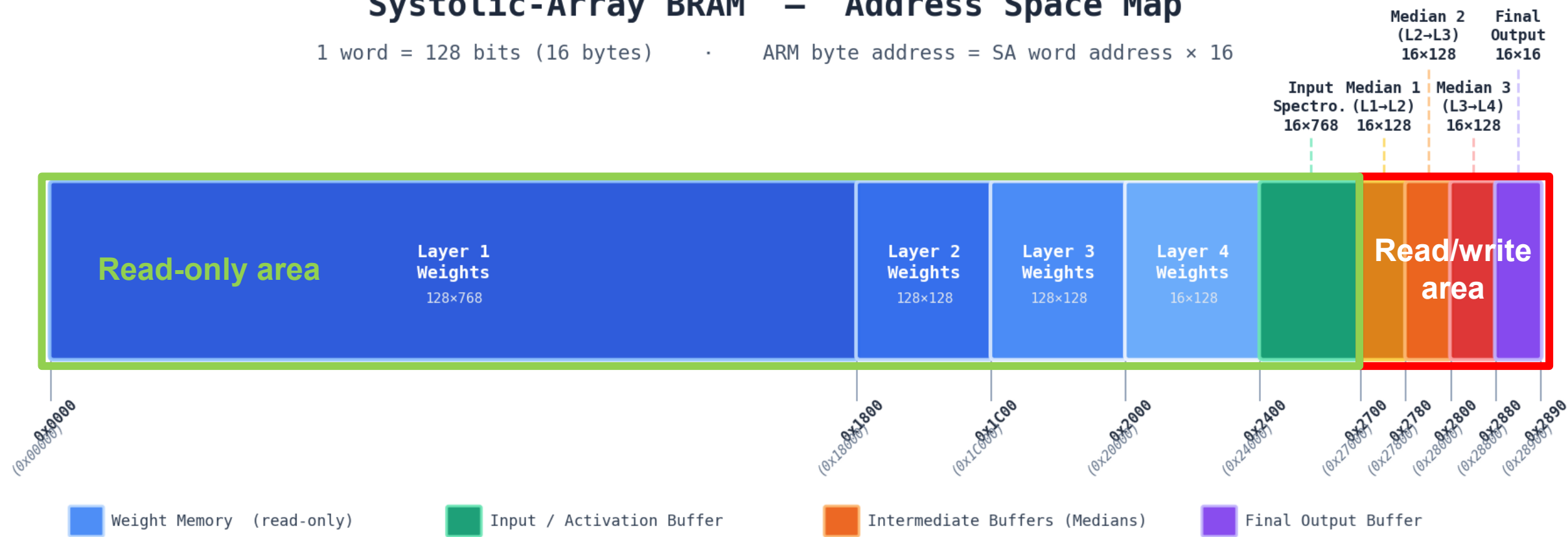
BRAM word address (14-bit space = 16 384 × 128-bit words)

0	0	1	2	3	4	5	6	7
	0	1	2	3	4	5	6	7

Material description – data structure

Systolic-Array BRAM – Address Space Map

1 word = 128 bits (16 bytes) · ARM byte address = SA word address × 16

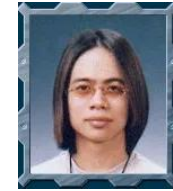


- 0x0000 – 0x26FF에 저장하면 **read only**데이터가 손상될 수 있습니다.
- 0x2700 – 0x287F는 **계산 중의 값을 저장하는 공간**이지만, (권장) 메모리는 0x3FFF까지 사용 가능합니다.
- 0x2880 – 0x288F는 모든 계산이 끝난 후 ARM이 **결과값을 읽어들이는 부분**입니다 (권장)

Material description

- Pre-tiled input/weights

- 난이도 조정을 위해 미리 타일링을 해 두었습니다, 따라서 출력하는 데이터 또한 타일링하기 수월합니다.
- 여러분이 할 일은 적절한 타일을 선택해 GEMM연산을 수행하고, 메모리의 적절한 위치에 저장해, 적절하게 최종 결과를 내는 것입니다.

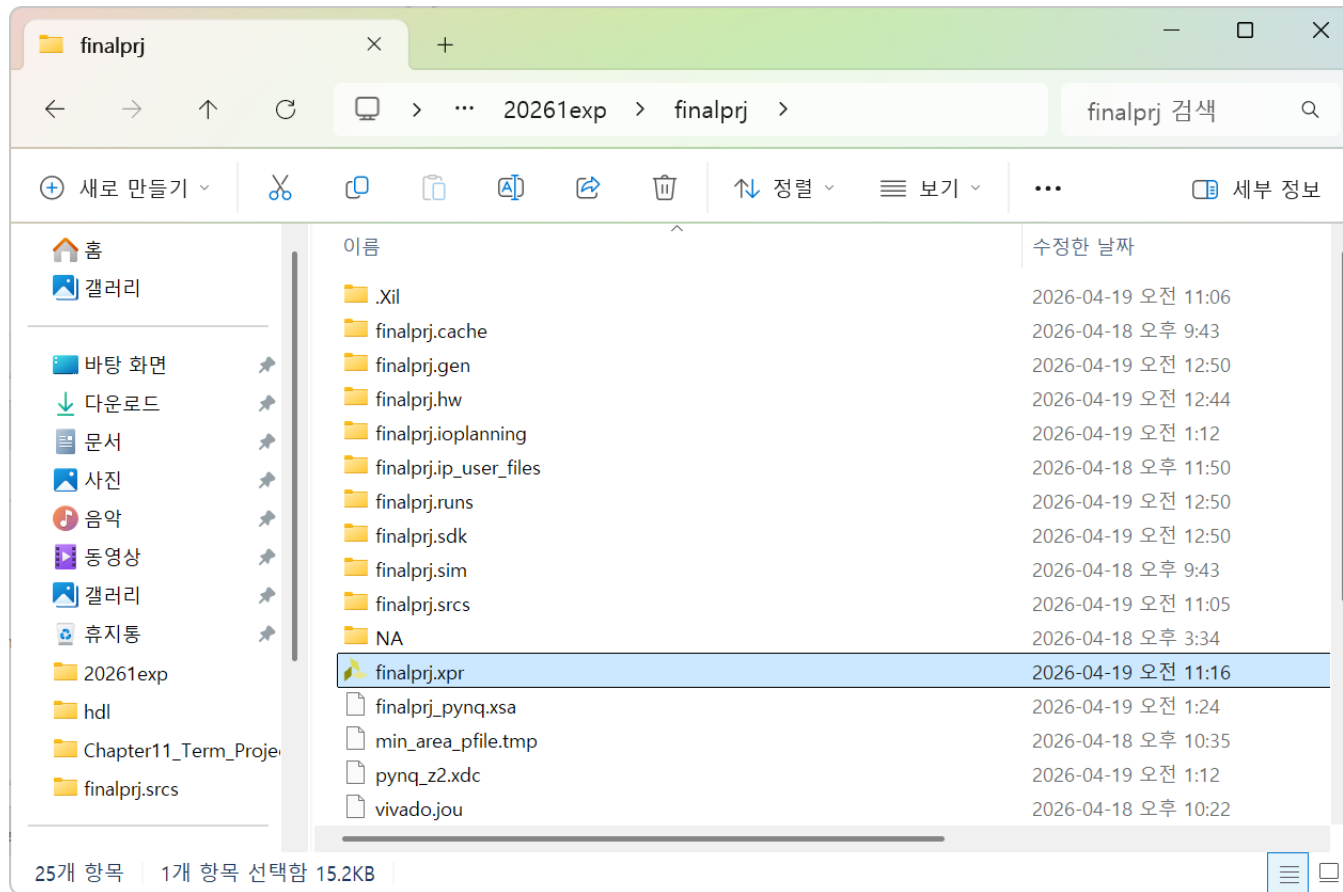




Vivado project



How to open the Vivado project



제공된 Vivado project를
열니다

How to open the Vivado project

The screenshot displays the Vivado 2022.1 IDE interface. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, View, and Help. The left sidebar contains the Project Manager, IP Integrator, Simulation, RTL Analysis, Synthesis, Implementation, and Program and Debug sections. The main workspace is divided into three panes: Sources, Properties, and Diagram. The Sources pane shows the project hierarchy with 'Utility Sources (1)'. The Properties pane is empty. The Diagram pane shows a block design diagram with components like 'rst_ps7_0_100M', 'processing_system7_0', 'ps7_0_axi_periph', and 'finalprj_wrapper_0'. The Tcl Console at the bottom shows the output of the 'run' command, including the path to the runme.log file and the time taken to execute the command.

finalprj - [D:/20261exp/finalprj/finalprj.xpr] - Vivado 2022.1

File Edit Flow Tools Reports Window View Help Quick Access

write_bitstream Complete

Default Layout

Flow Navigator

BLOCK DESIGN - design_1

Sources Design Signals Board

Utility Sources (1)

Hierarchy IP Sources Libraries Compile Order

Properties

Select an object to see properties

Diagram Address Editor Address Map

Default View

Diagram components: rst_ps7_0_100M, processing_system7_0, ps7_0_axi_periph, finalprj_wrapper_0, finalprj_wrapper_v1_0.

Tcl Console

Messages Log Reports Design Runs

INFO: [IP_Flow 19-4993] Using cached IP synthesis design for IP design_1_auto_pc_0, cache-ID = 42ad08fd9081ea99; cache size = 441.700 MB.

INFO: [IP_Flow 19-4993] Using cached IP synthesis design for IP design_1_finalprj_wrapper_0_1, cache-ID = 5e00b7b3b261b; cache size = 441.700 MB.

[Sun Apr 19 11:06:12 2026] Launched synth_1...

Run output will be captured here: D:/20261exp/finalprj/finalprj.runs/synth_1/runme.log

[Sun Apr 19 11:06:12 2026] Launched impl_1...

Run output will be captured here: D:/20261exp/finalprj/finalprj.runs/impl_1/runme.log

launch_runs: Time (s): cpu = 00:00:06 ; elapsed = 00:00:14 . Memory (MB): peak = 6528.727 ; gain = 0.000

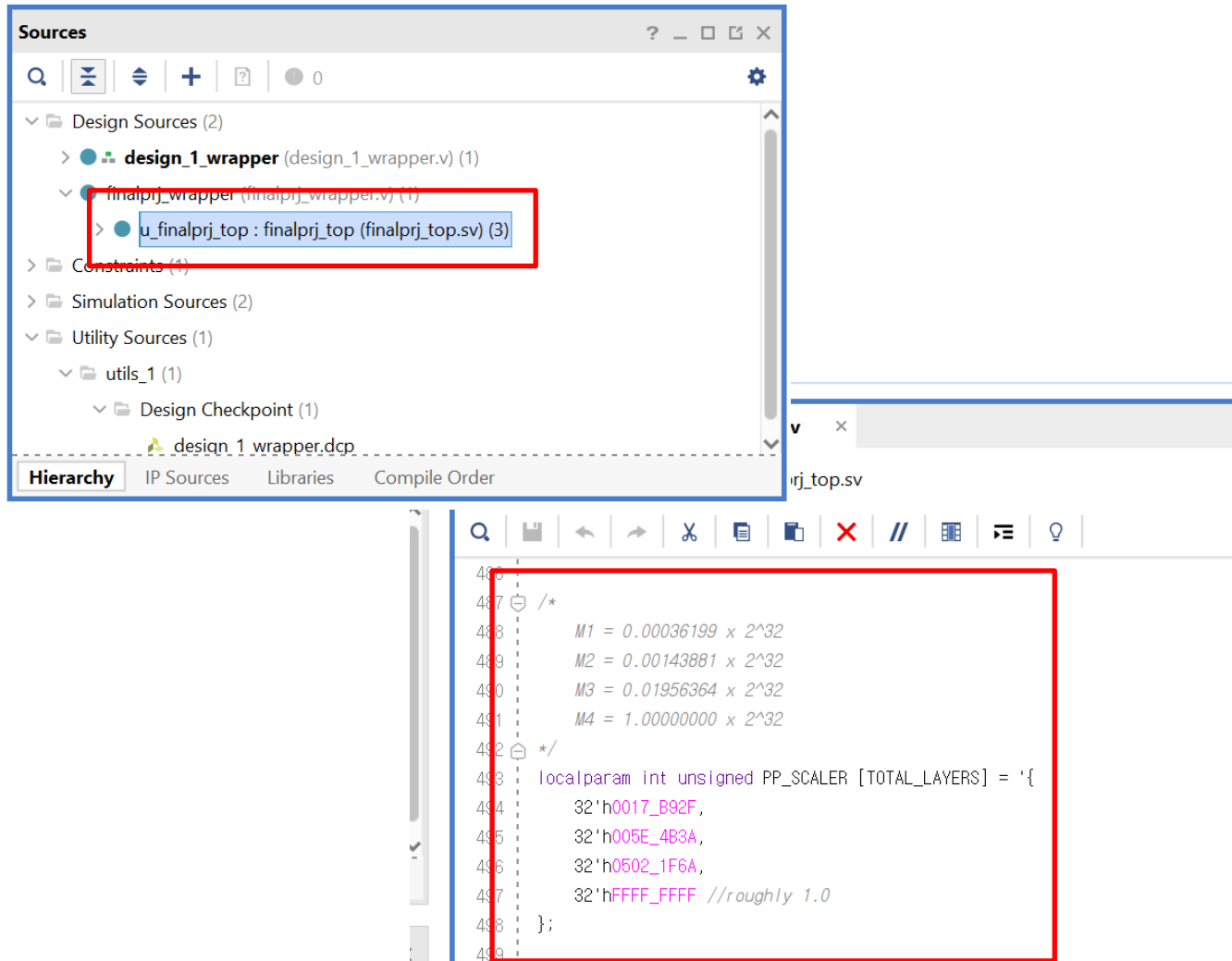
regenerate_bd_layout

regenerate_bd_layout

Type a Tcl command here

제공된 Vivado project가 열린 모습입니다.

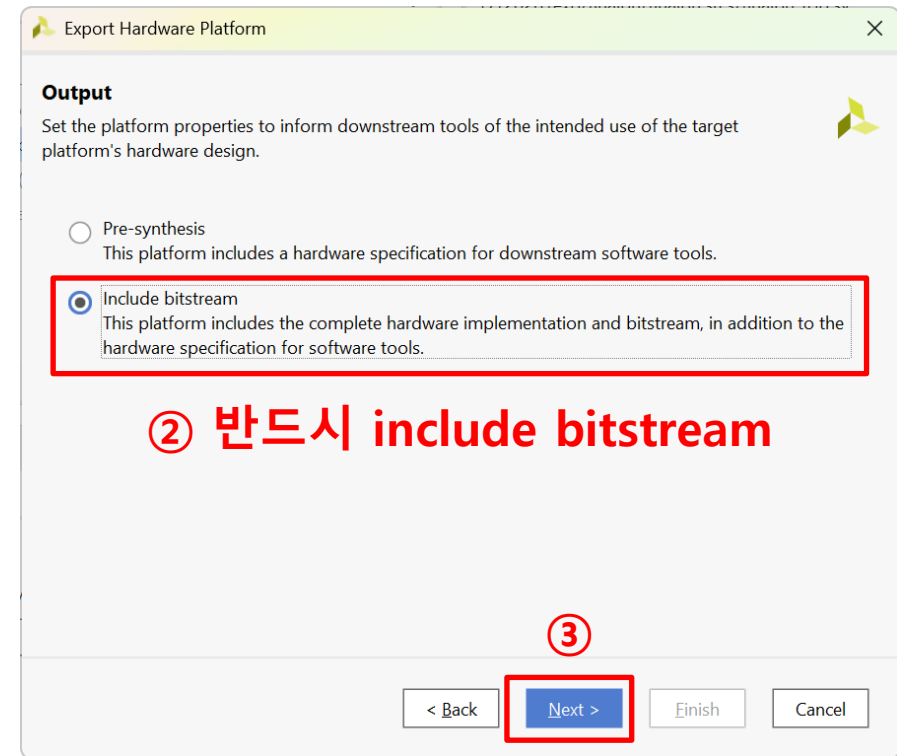
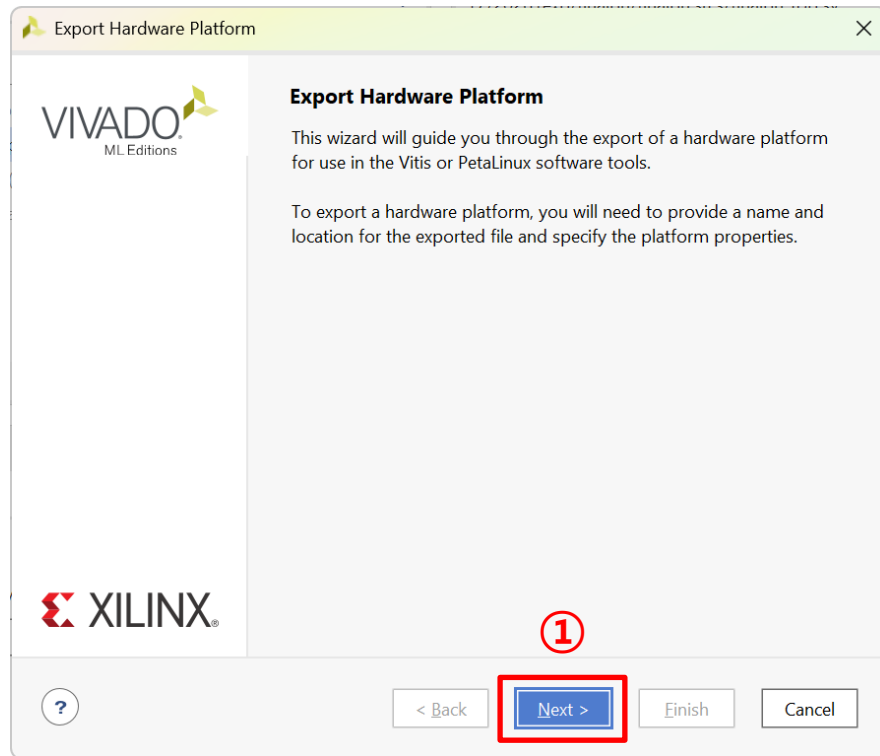
How to open the Vivado project



finalprj_wrapper.vv의 내용,
BRAM_TDP 모듈의 내용,
finalprj_top.vv의 포트는 변경하면
안 됩니다.

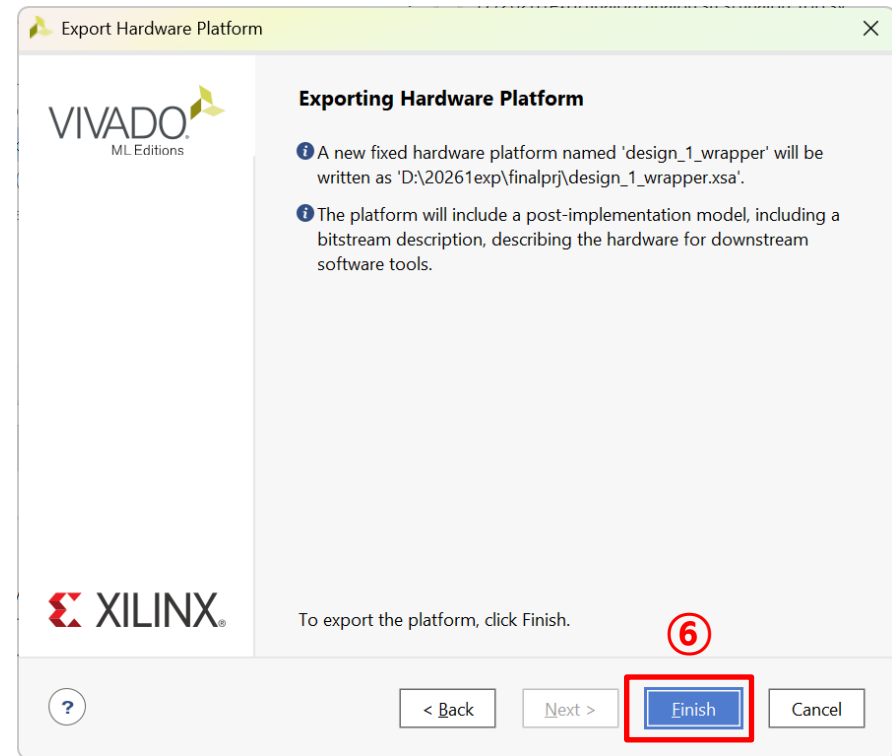
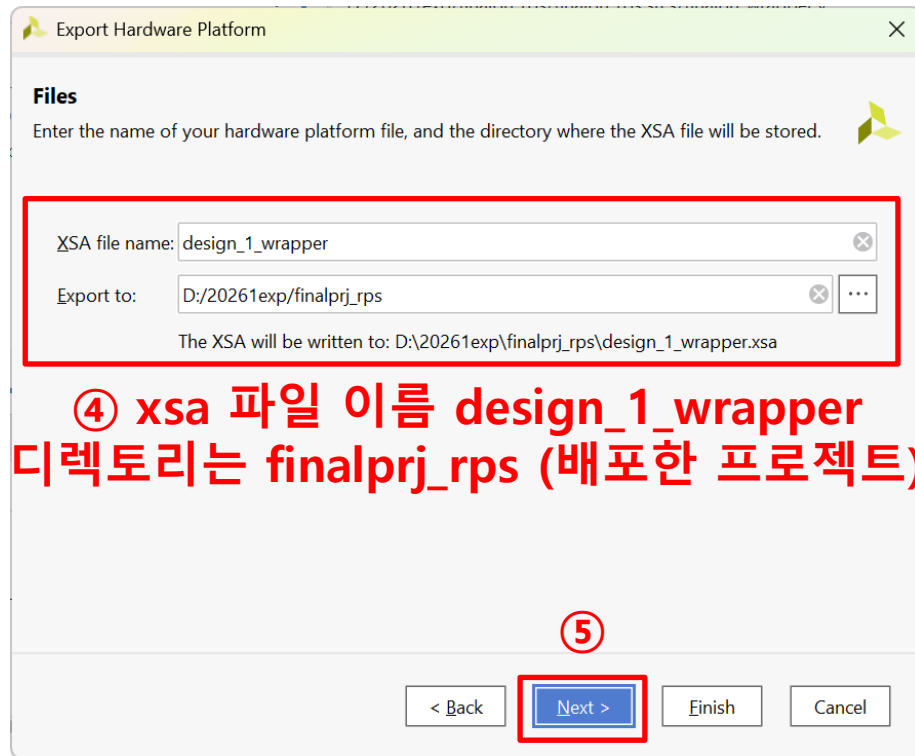
기본적으로 제공한 샘플 코드와
고정소수점 상수값 (unsigned) 이
있습니다.

How to export a hardware



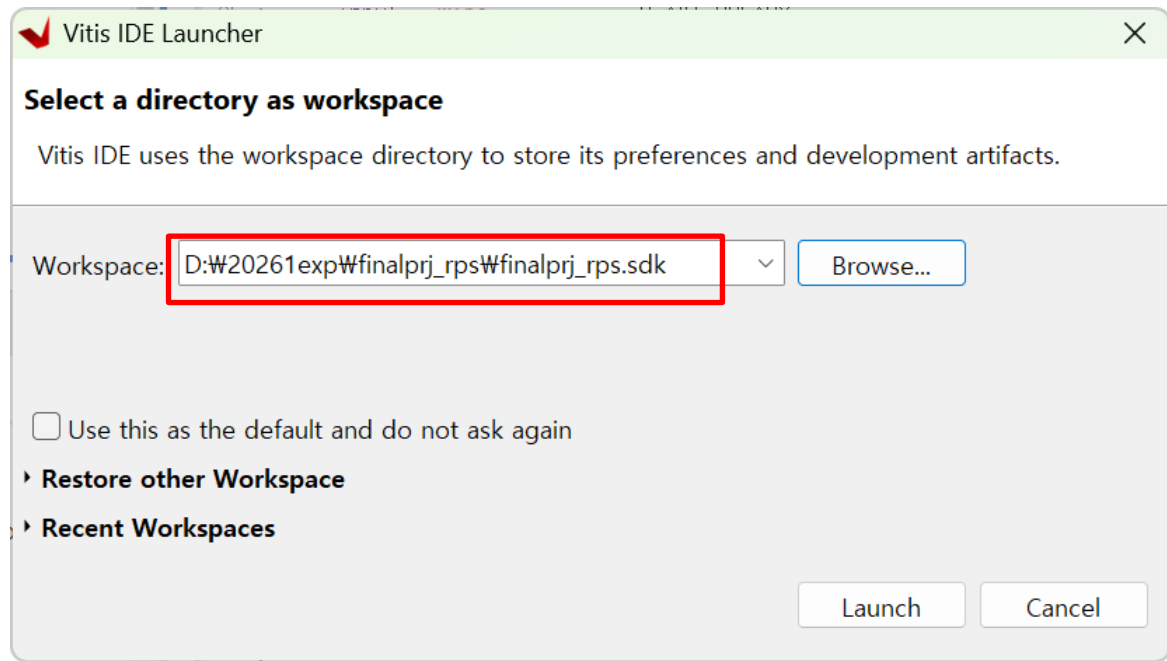
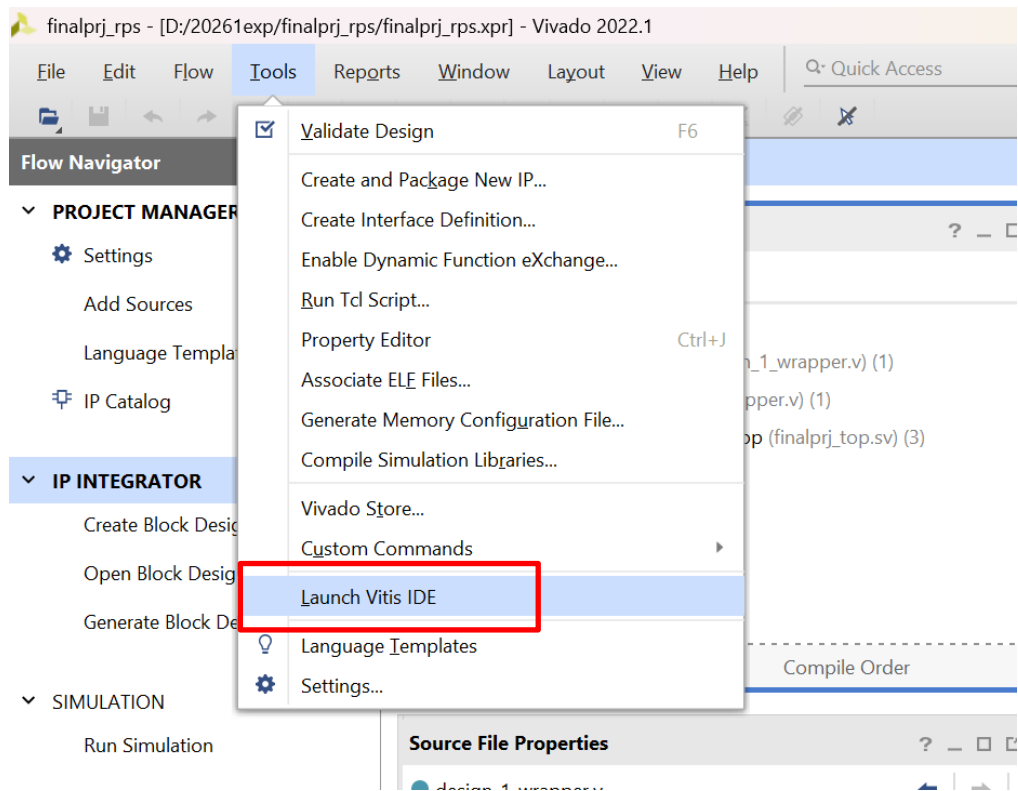
Vitis (ARM SDK) 가 현재 하드웨어 구성을 알 수 있게 하려면 hardware file (.xsa) 를 export해야 합니다.

How to export a hardware



Vitis (ARM SDK) 가 현재 하드웨어 구성을 알 수 있게 하려면 hardware file (.xsa) 를 export해야 합니다.

How to launch Vitis



Vitis를 실행할 때는 skeleton 하위 폴더인 "finalprj_rps.sdk" 폴더를 Vitis workspace로 지정합니다.

How to launch Vitis

The screenshot displays the Vivado 2022.1 Vitis interface for a project named 'finalprj_rps'. The interface is divided into several panes:

- Flow Navigator:** Shows the project workflow with steps like Settings, Add Sources, Language Templates, IP Catalog, IP INTEGRATOR, SIMULATION, RTL ANALYSIS, SYNTHESIS, IMPLEMENTATION, and PROGRAM AND DEBUG.
- Project Manager:** Displays the project hierarchy with 'Sources' (Constraints (1), Utility Sources (1)) and 'Properties'.
- Project Summary:** Provides an overview of project settings, including project name, location, product family, and target language.
- Synthesis and Implementation:** Shows the status of synthesis and implementation, including messages, active runs, and DRC violations.
- Design Runs:** A table showing the results of various design runs, including synthesis and implementation.

The 'Design Runs' table is as follows:

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start
synth_1 (active)	constrs_1	synth_design Complete!												0	0	0	0	0	4/20/26, 8:50 PM
impl_1	constrs_1	write_bitstream Complete!	2.191	0.000	0.025	0.000		0.000	1.708	0	2 Warn			657	739	64	0	0	4/20/26, 8:51 PM
Out-of-Context Module Runs																			
design_1		Submodule Runs Complete																	4/20/26, 7:17 PM

Vitis 화면

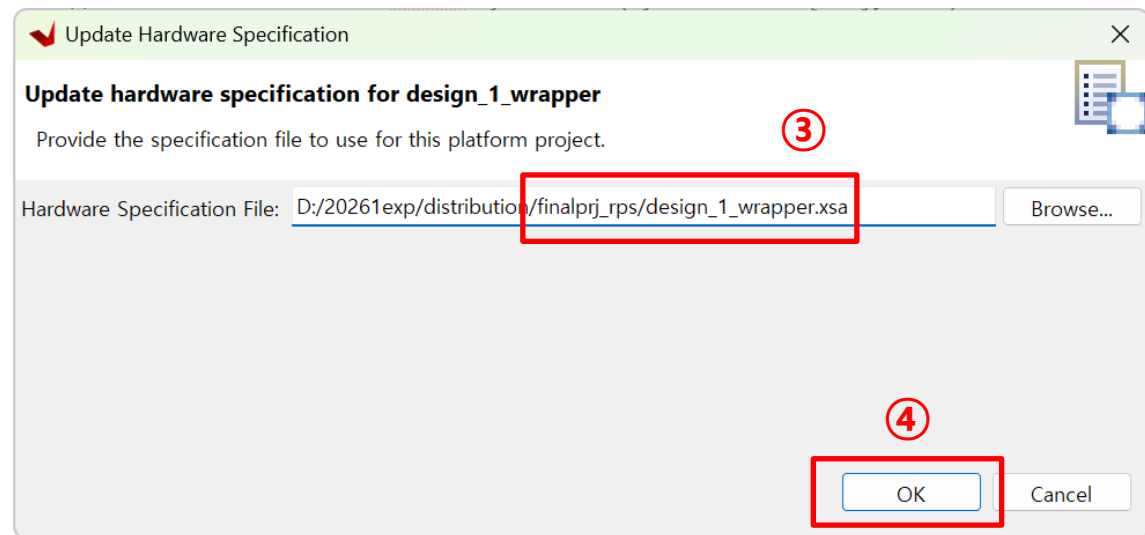
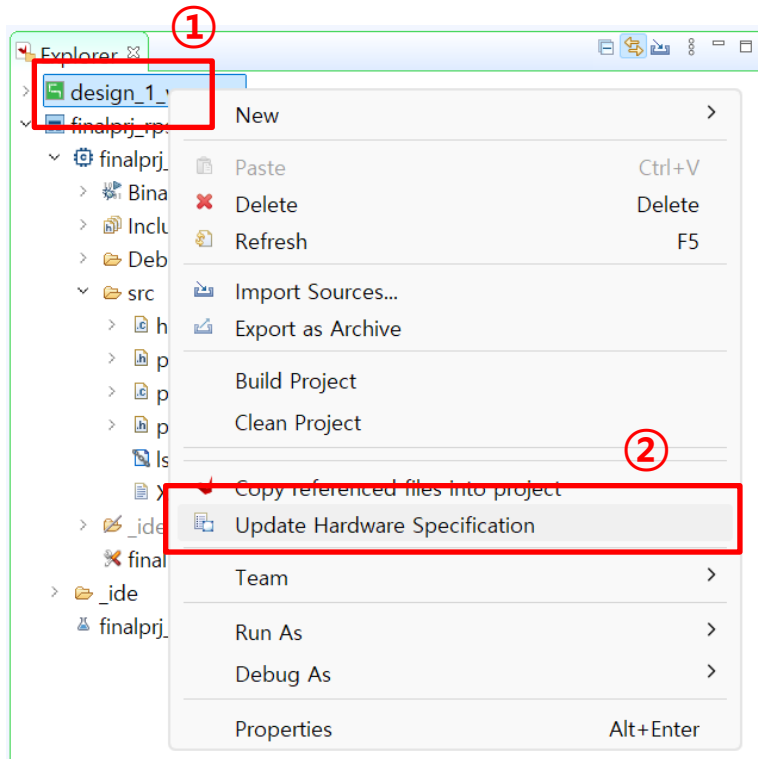
How to launch Vitis

```
62 #define INTC_DEVICE_INT_ID 61 // ID 61
63
64 #define OUTPUT_BRAM_OFFSET 0x28800 // Byte offset from PL base address to the 16x16 result matrix
65 #define NUM_CLIPS 16 // Number of audio clips (rows in output matrix)
66 #define NUM_CLASSES 9 // Number of valid output classes (columns 0..8)
67 #define OUTPUT_COLS 16 // Total columns in the output matrix
68
69 int SetUpInterruptSystem(XScuGic *XScuGicInstancePtr);
70 void DeviceDriverHandler(void *CallbackRef):
```

OUTPUT_BRAM_OFFSET 상수를 변경하여 16x16 출력을 읽어들이기 시작 주소를 지정 가능합니다.
권장은 0x28800이지만, 최종 출력 주소를 바꾸고 싶은 분은 C코드의 이쪽을 변경하세요.

시스톨릭 어레이 코어에서는 1워드=128비트지만 ARM에서 1워드=32비트이고, 8비트 단위로 어드레스가 증가하므로, 시스톨릭 어레이가 보는 주소 × 16을 하면 ARM쪽 어드레스를 구할 수 있습니다.

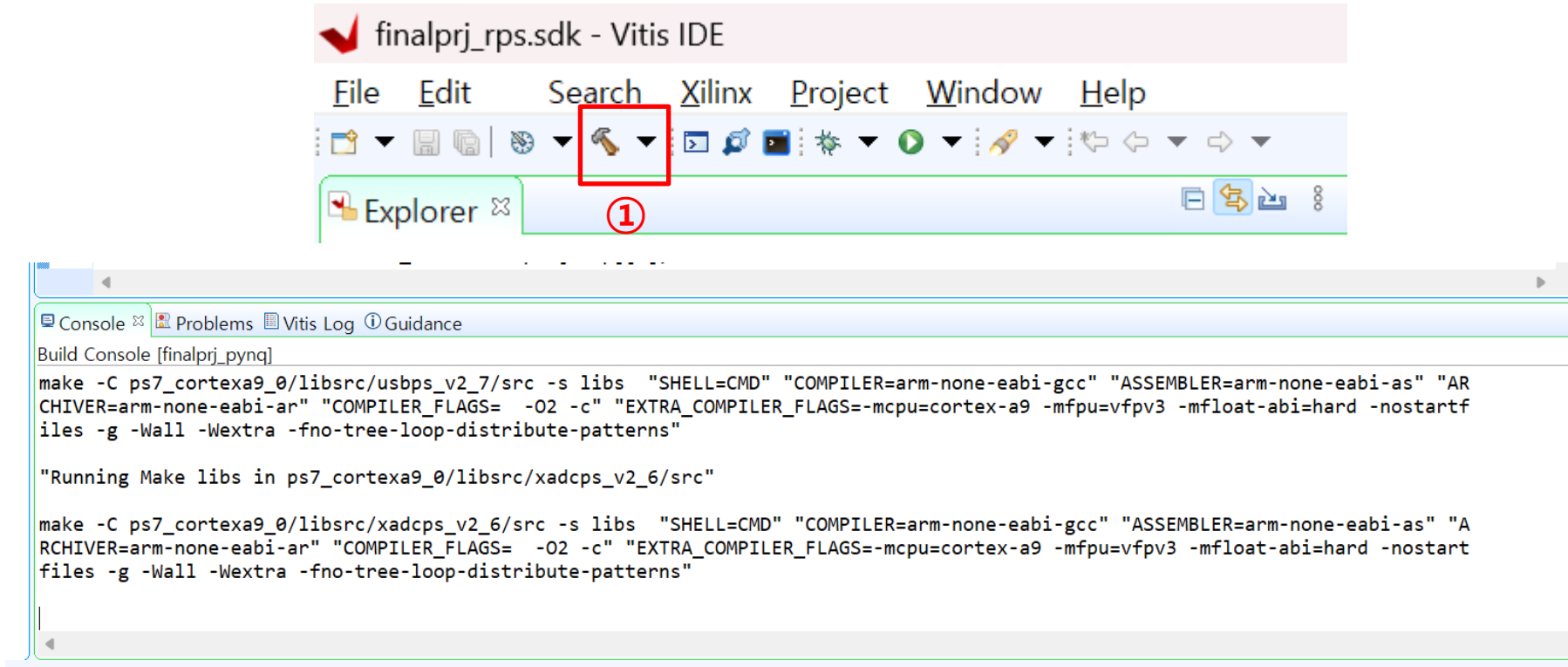
How to compile sources on Vitis



새롭게 **bitstream**을 생성했으면 반드시 다음 갱신 절차를 거쳐야 합니다.

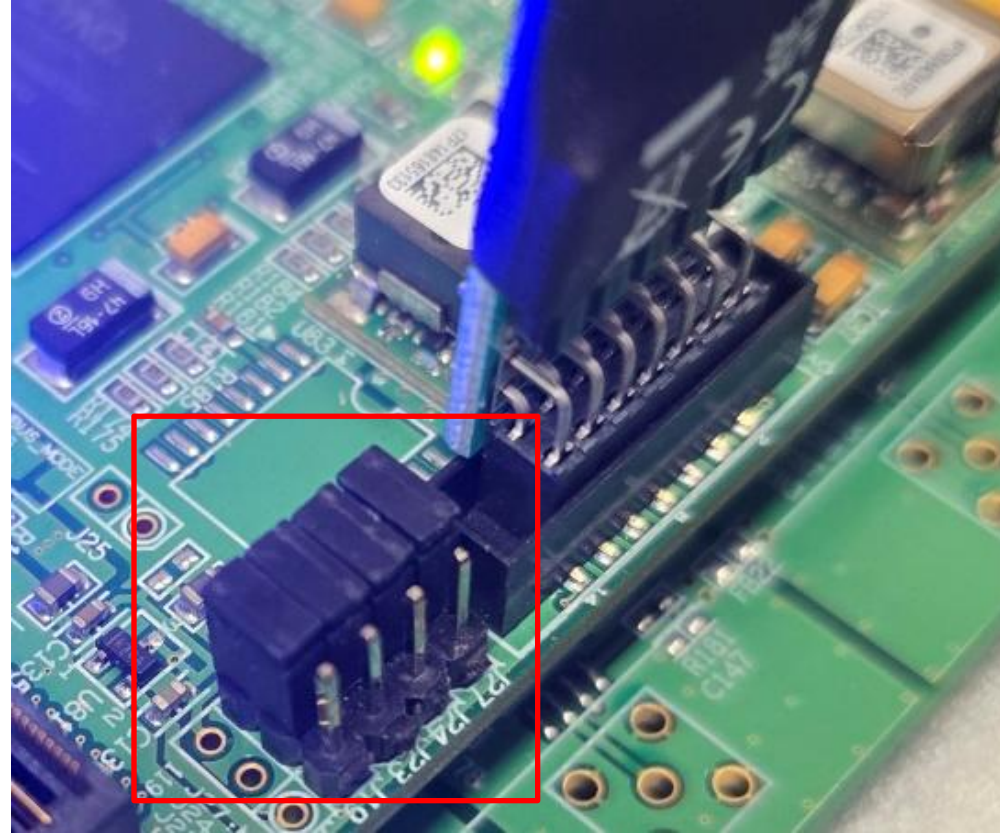
초록색 아이콘 오른쪽 클릭 -> Update Hardware Specification -> 출력한 xsa파일 선택 -> OK

How to compile sources on Vitis



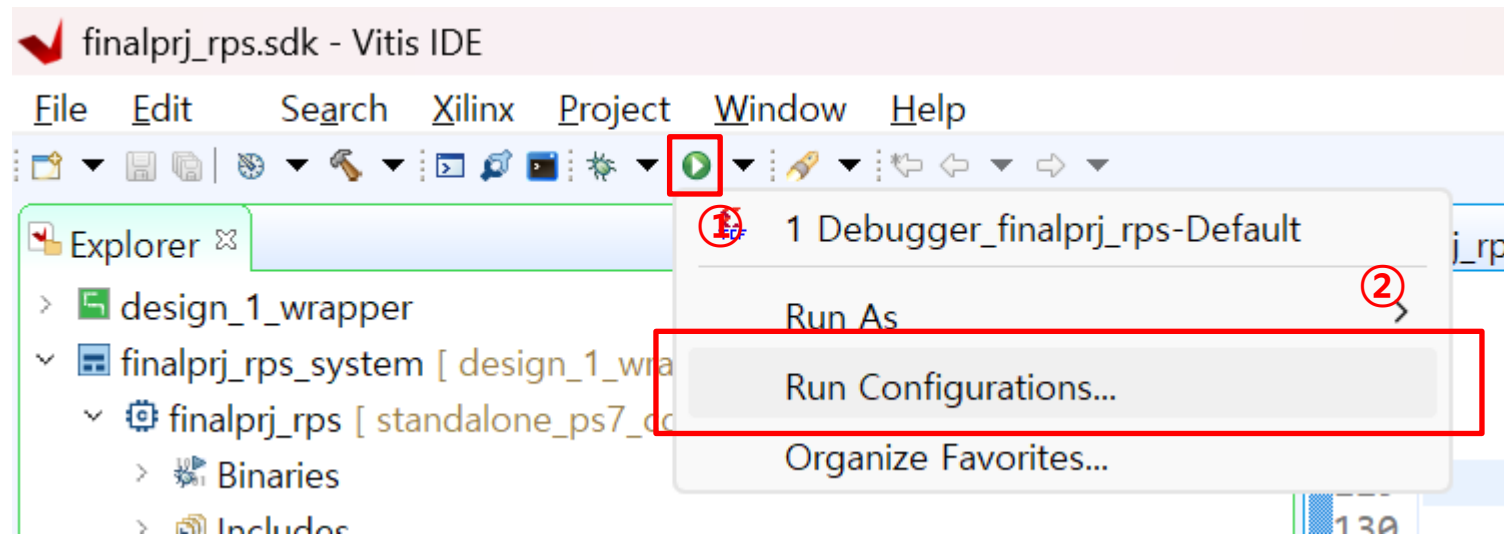
망치 아이콘을 누르면 프로젝트 컴파일이 시작됩니다 (자동으로 진행됨)
모두 끝나면 **Build Finished** 라고 뜹니다.

How to upload a program via JTAG



보드 JTAG포트 아랫쪽의 점퍼 4개를 모두 왼쪽으로 위치하게 바꿉니다.

How to upload a program via JTAG



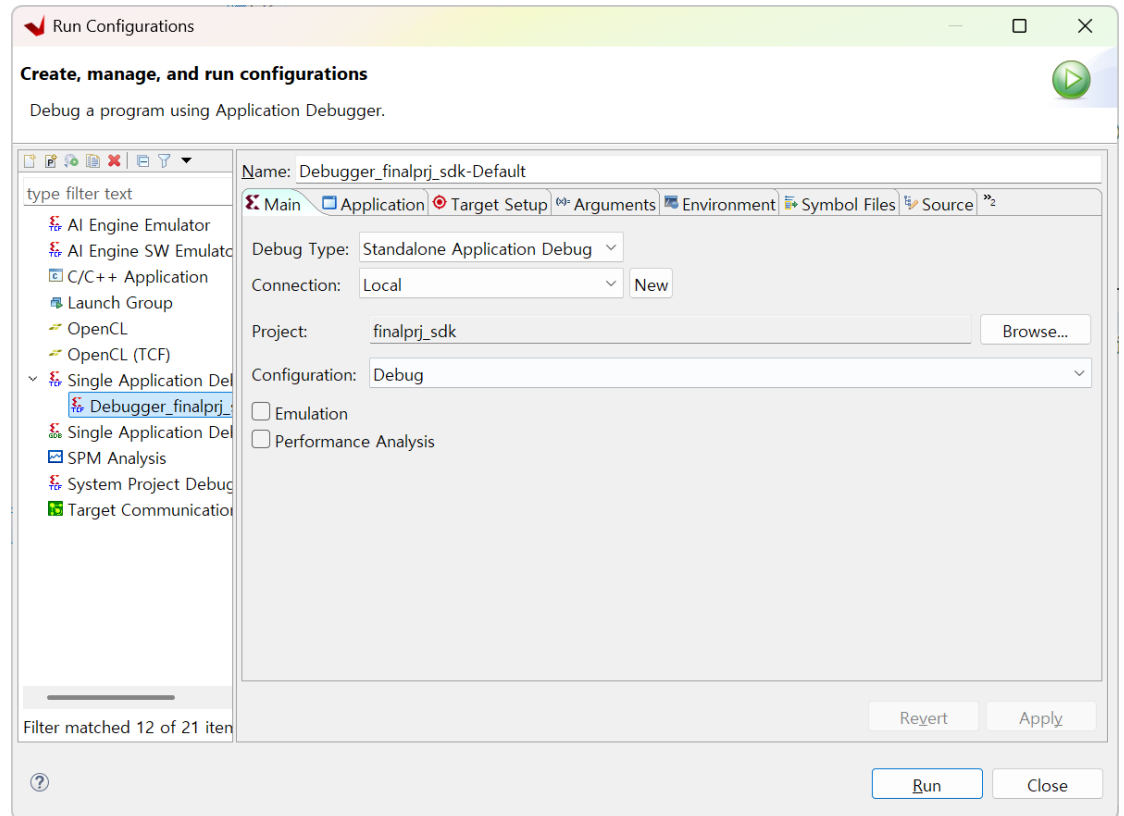
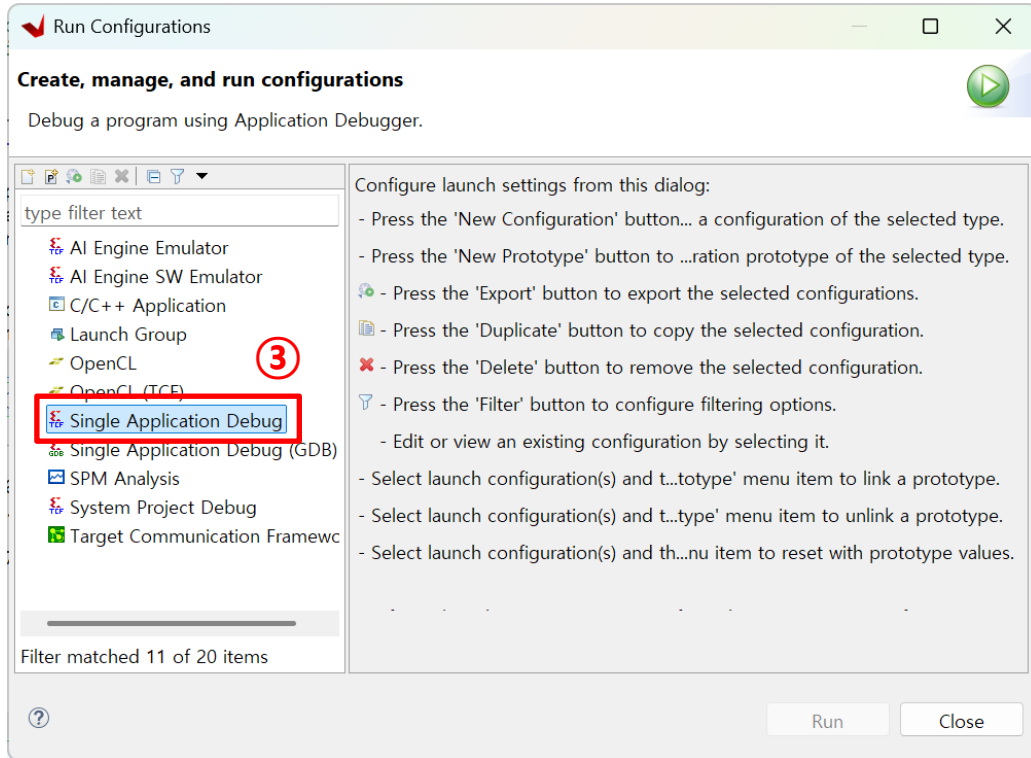
Vitis에서 처음으로 FPGA에 데이터를 올리는 경우, Run configuration을 새로 세팅해줘야 합니다.

녹색 ▶ 버튼 오른쪽 ▼를 누른 후, Run Configurations...를 누릅니다.

(두번째 이후부터는 Debugger...가 등록되어 있어서 알 수 있습니다.)

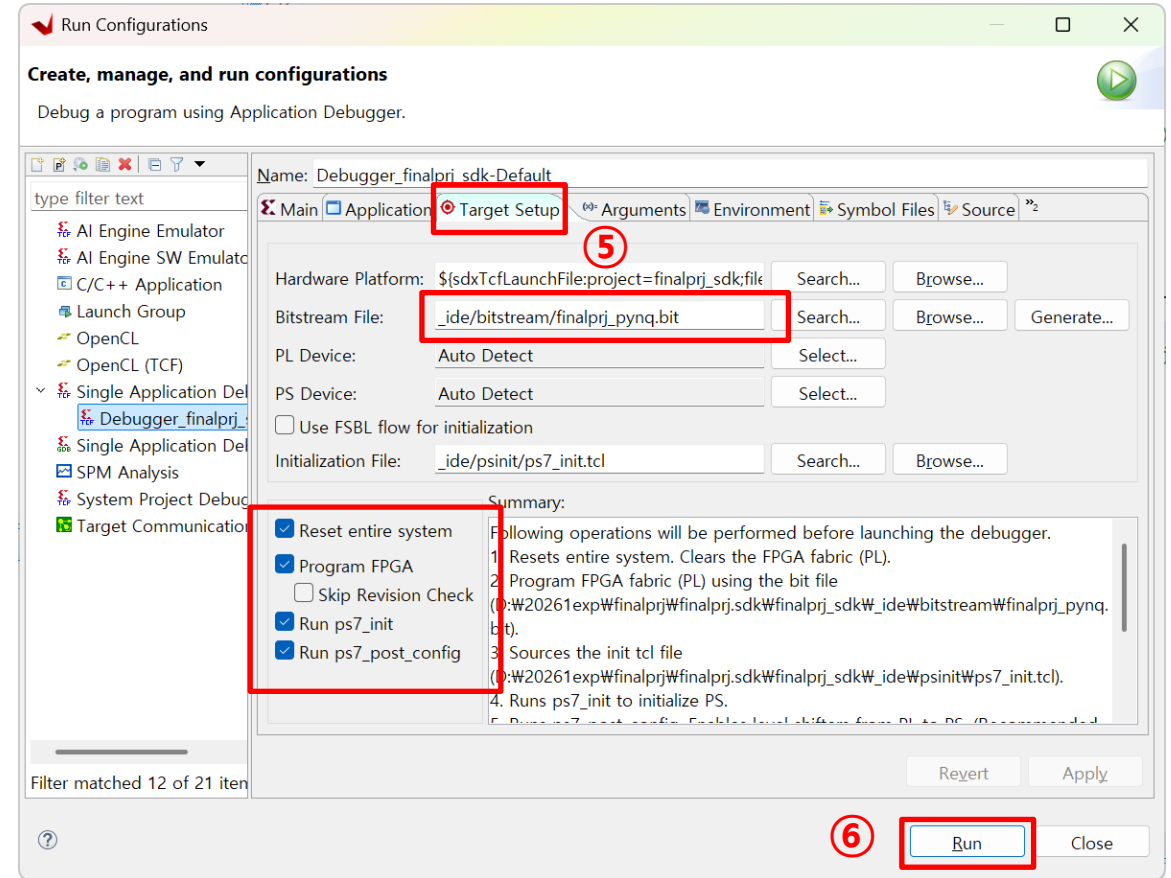
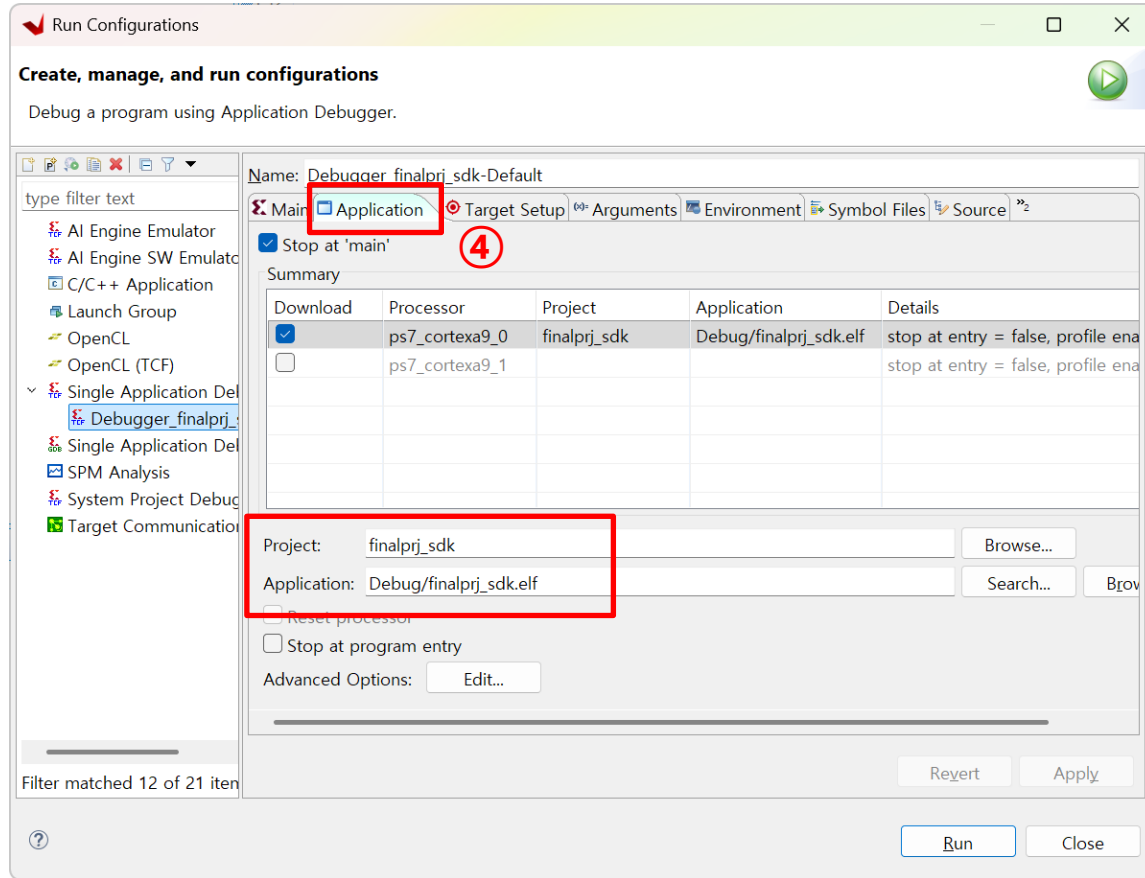
※ Verilog코드 수정 후, 비트스트림을 갱신하고, 다시 데이터를 올리는 경우에는 **녹색 ▶ 버튼만** 누릅니다.

How to upload a program via JTAG



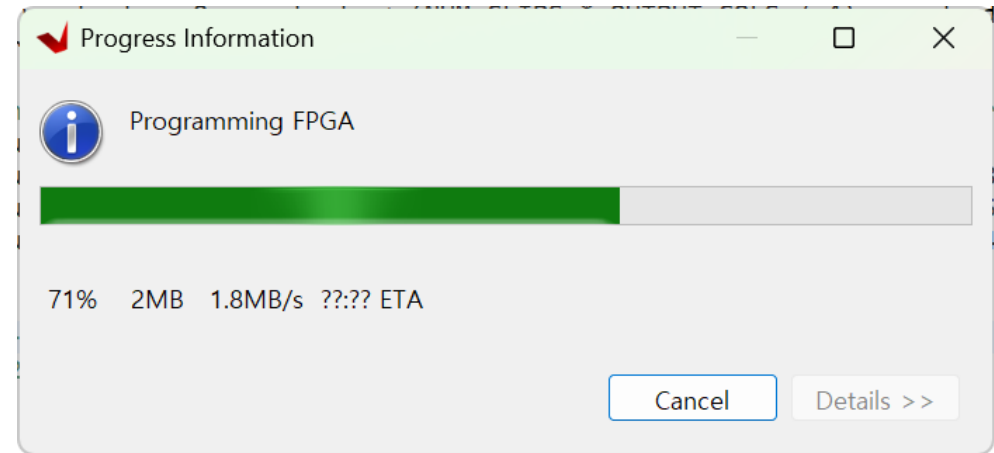
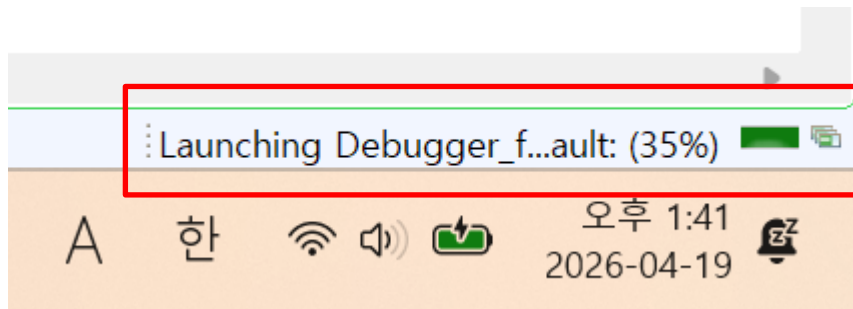
Single Application Debug 더블클릭하면 오른쪽 화면이 뜹니다.

How to upload a program via JTAG



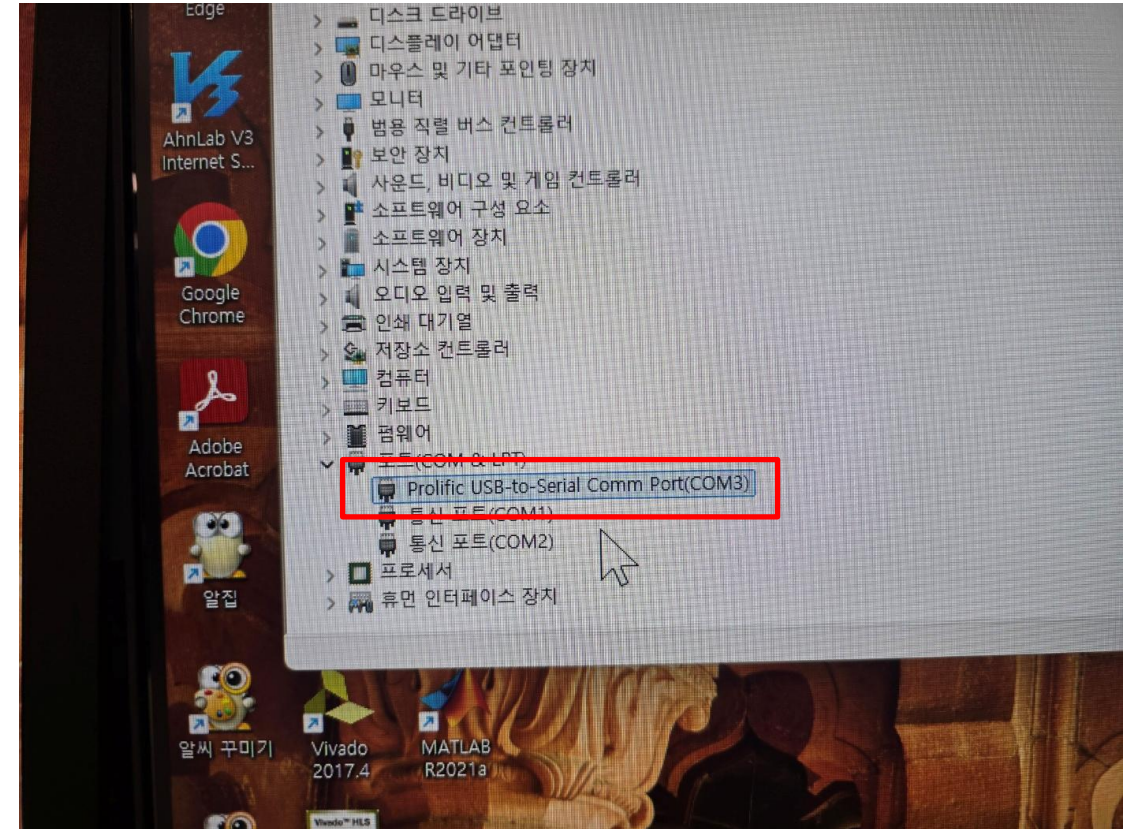
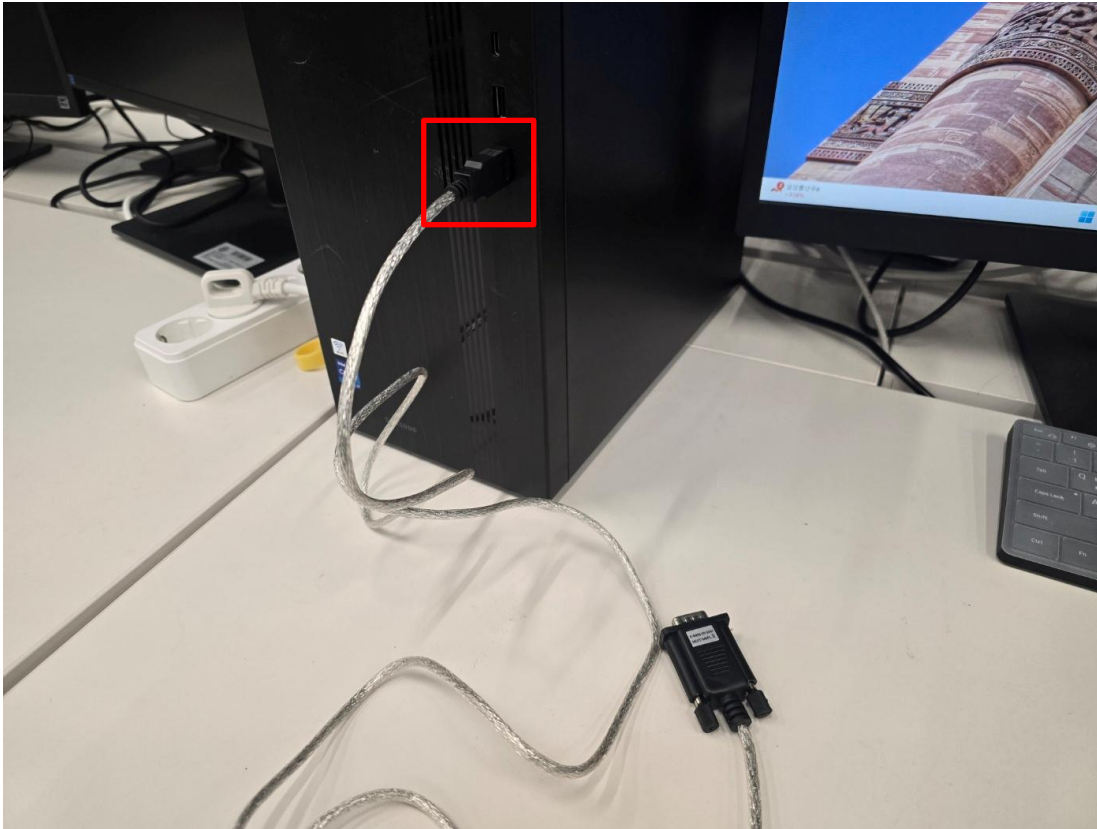
Application 탭에서 .elf 파일이 있나 확인합니다. **컴파일 먼저** 하지 않으면 이게 생기지 않습니다.
Target Setup 탭에서 .bit 파일이 등록되어 있는지, 사진 체크박스처럼 체크가 되어있는지 확인합니다.
모두 완료되었으면 Run 클릭

How to upload a program via JTAG



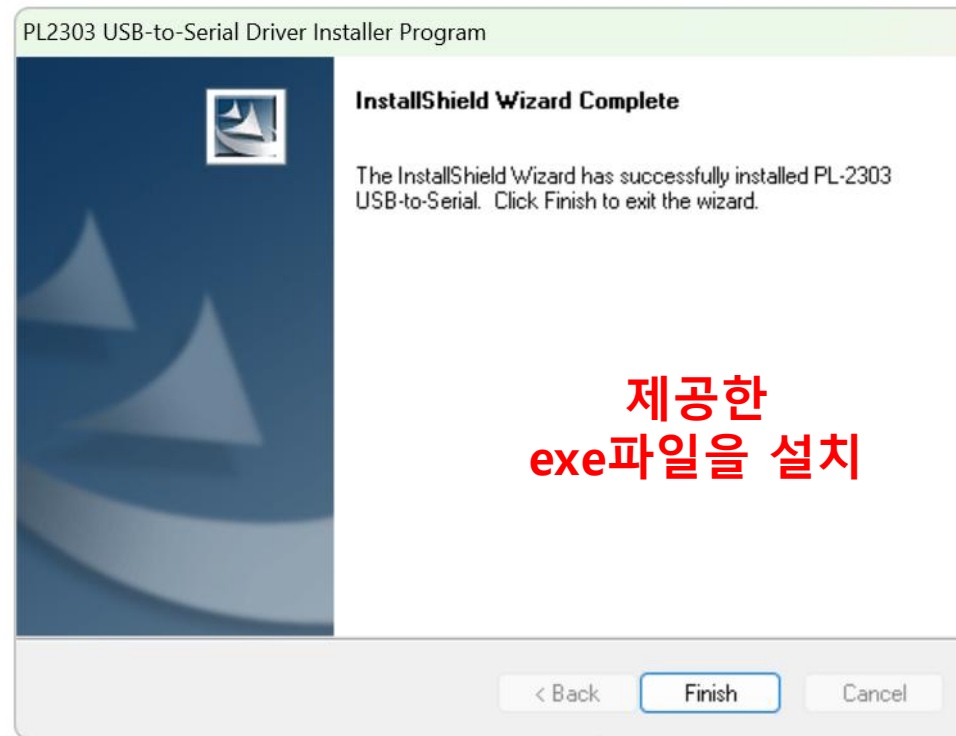
문제 없으면 오른쪽 아래 Launching Debugger 메시지가 뜨고, Programming FPGA창이 뜹니다.

How to connect an FPGA to a terminal via UART



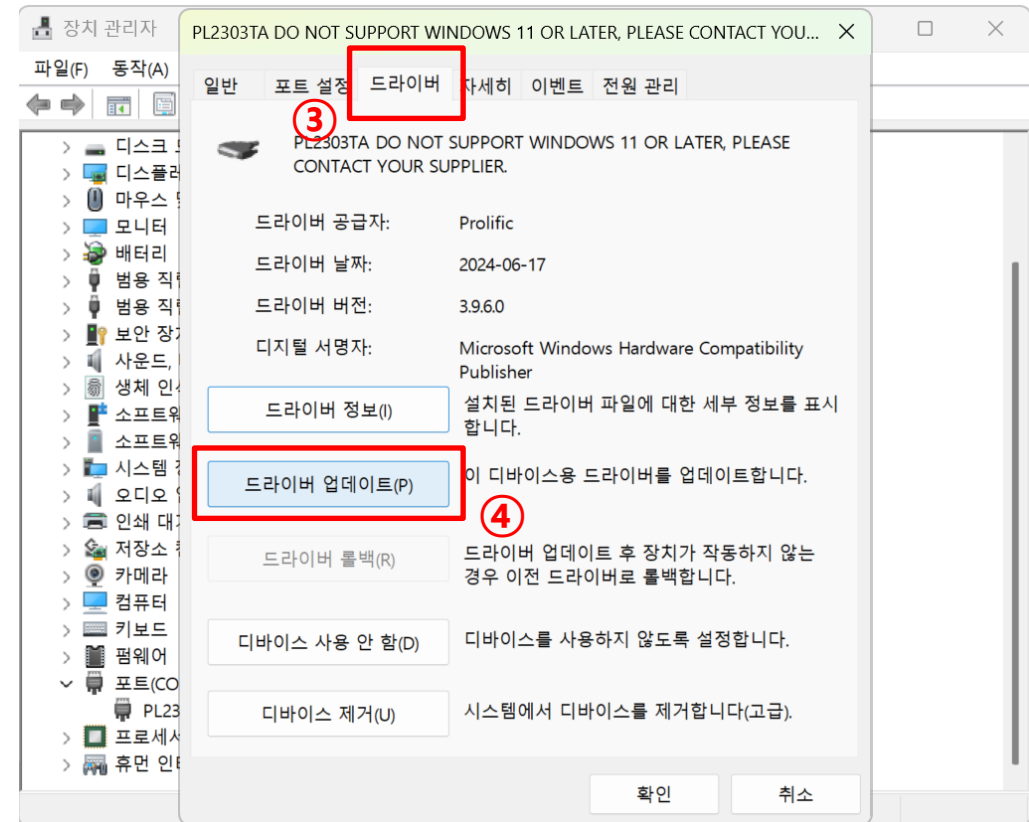
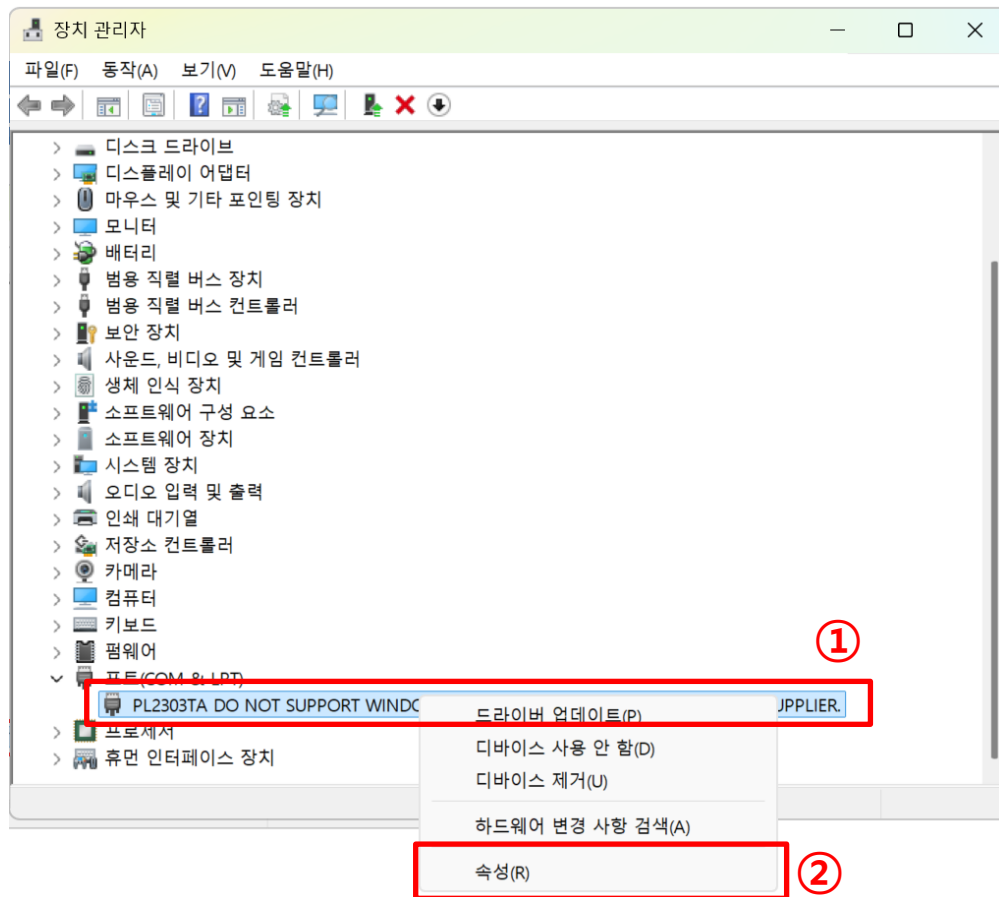
(실습실 데스크탑) USB 포트에 컨버터를 꽂은 후 최장 5분동안 기다립니다 (자동으로 드라이버 설치됨)

How to connect an FPGA to a terminal via UART



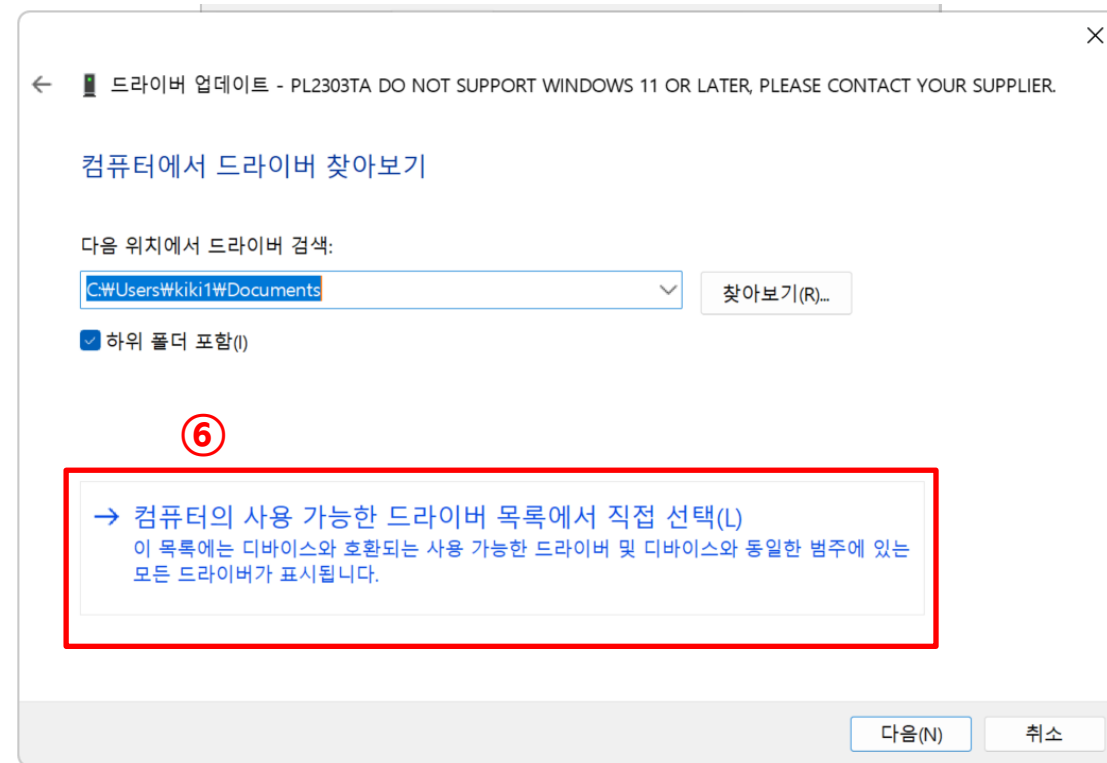
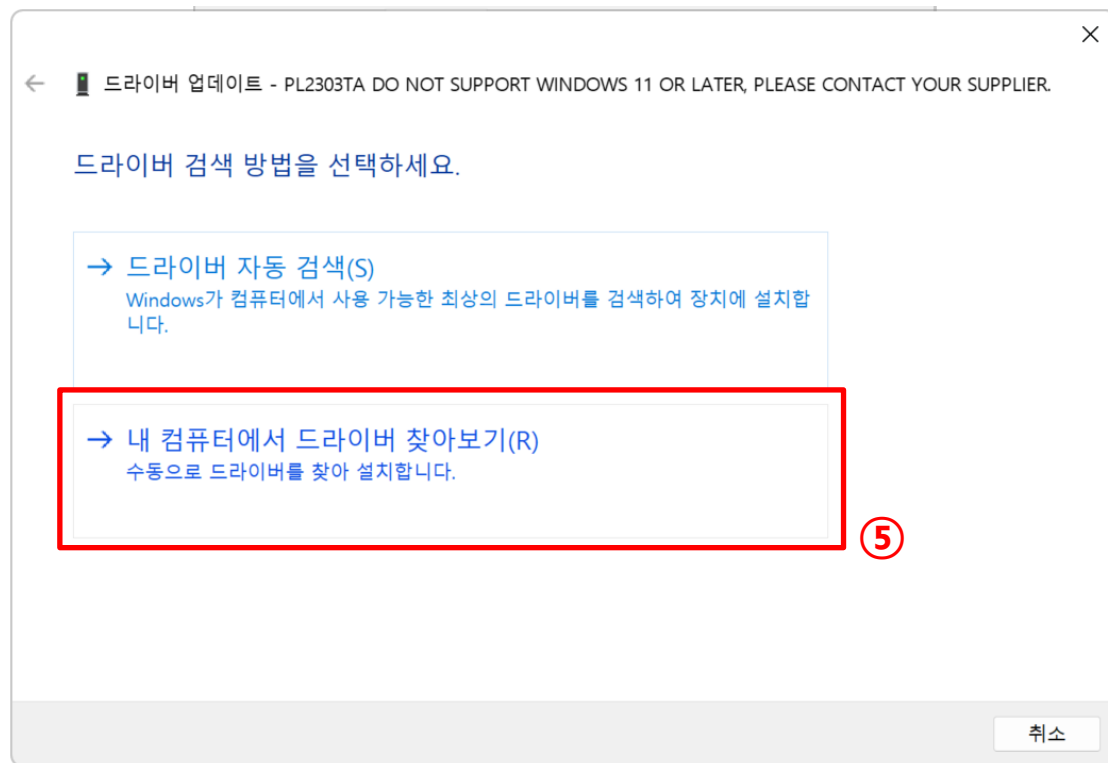
(개인 랩탑) PL2303TA의 경우 드라이버를 구버전으로 바꿔야 Windows 11에서 인식됩니다.

How to connect an FPGA to a terminal via UART



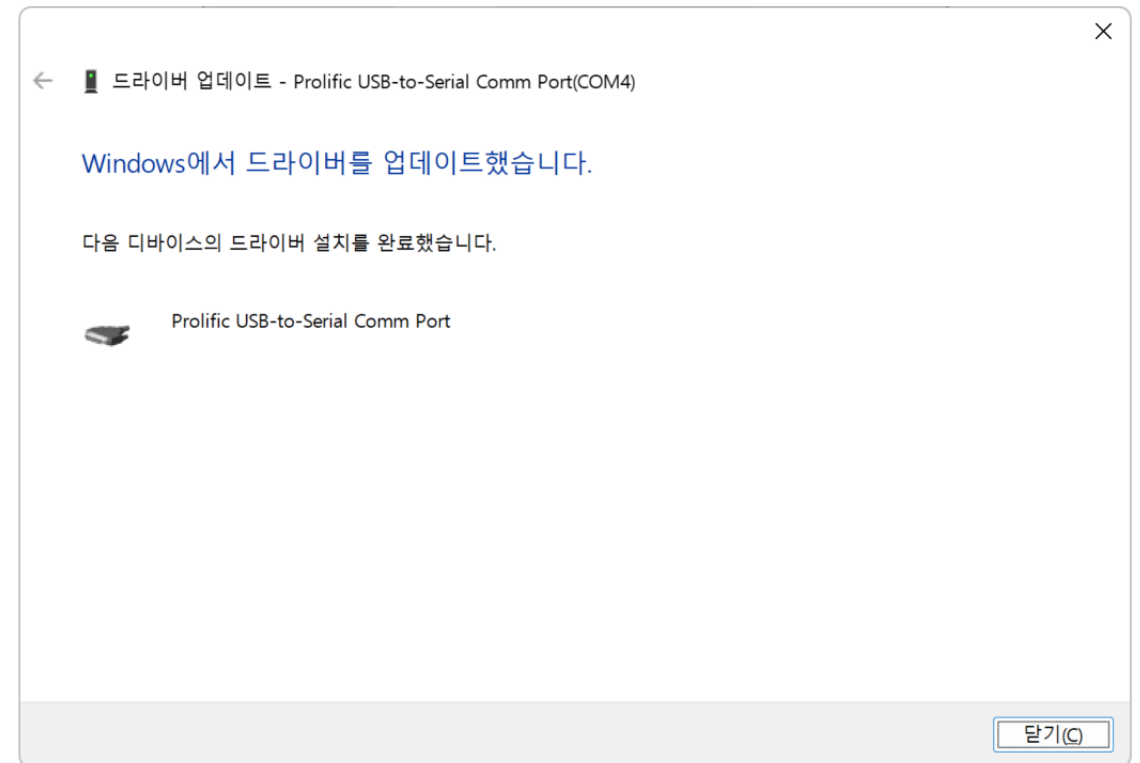
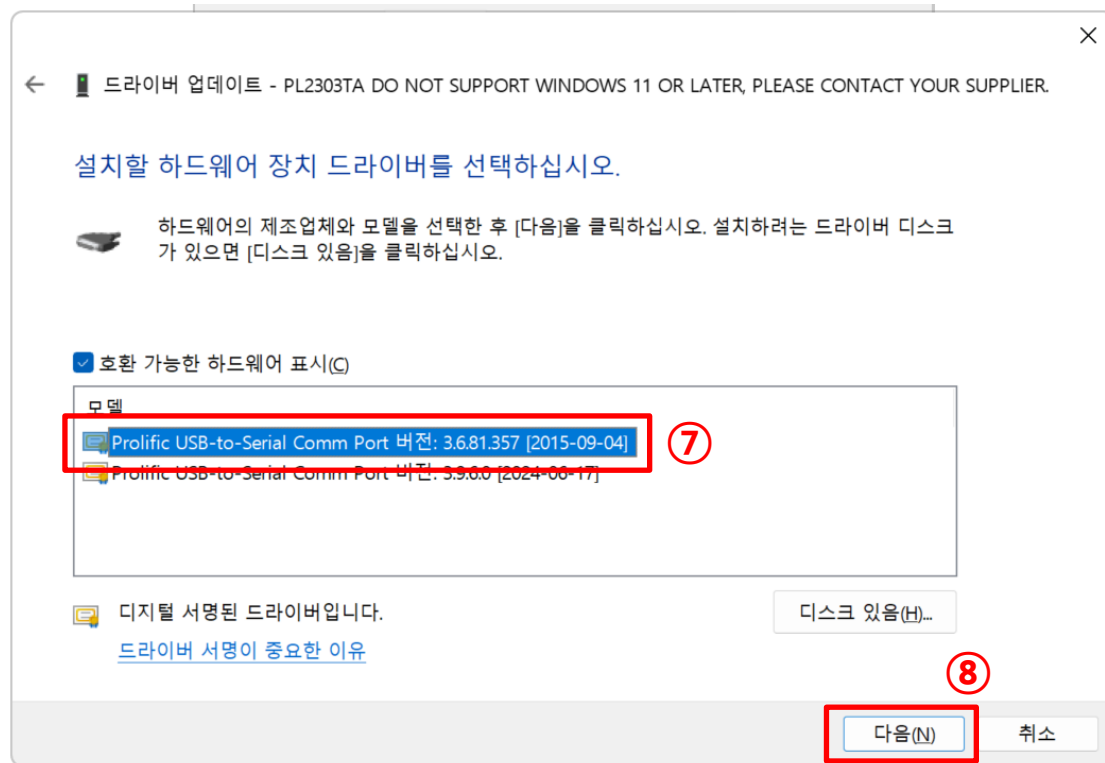
(개인 랩탑) PL2303TA의 경우 드라이버를 구버전으로 바꿔야 Windows 11에서 인식됩니다.

How to connect an FPGA to a terminal via UART



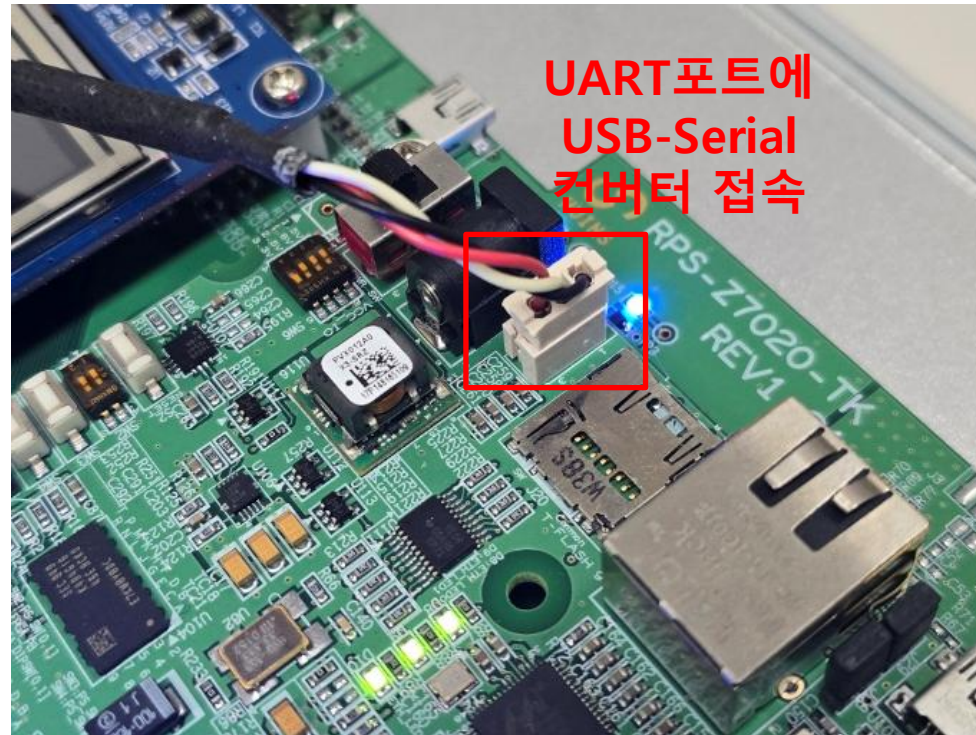
(개인 랩탑) PL2303TA의 경우 드라이버를 구버전으로 바꿔야 Windows 11에서 인식됩니다.

How to connect an FPGA to a terminal via UART



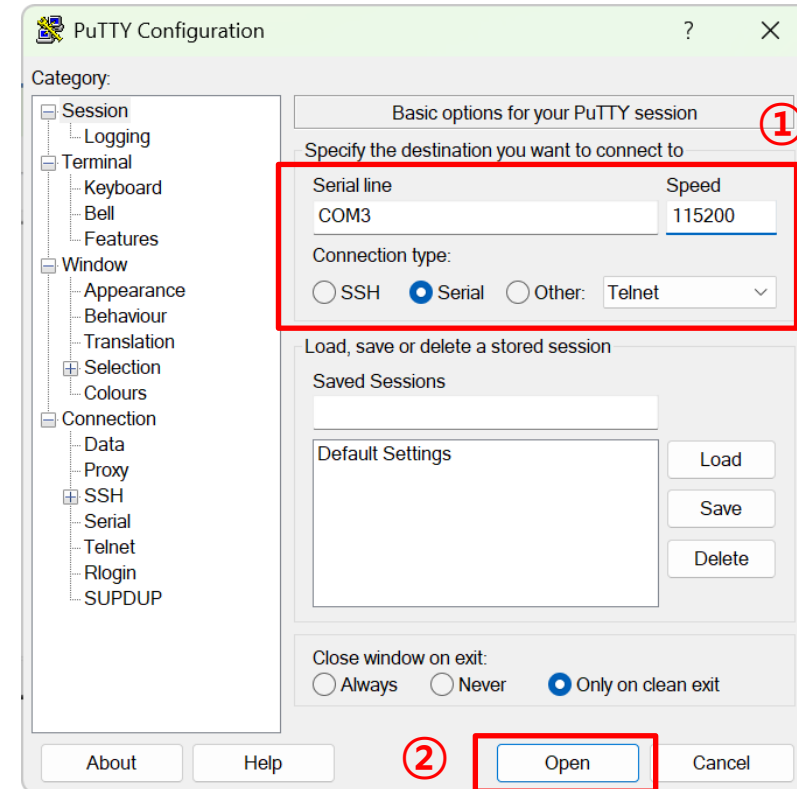
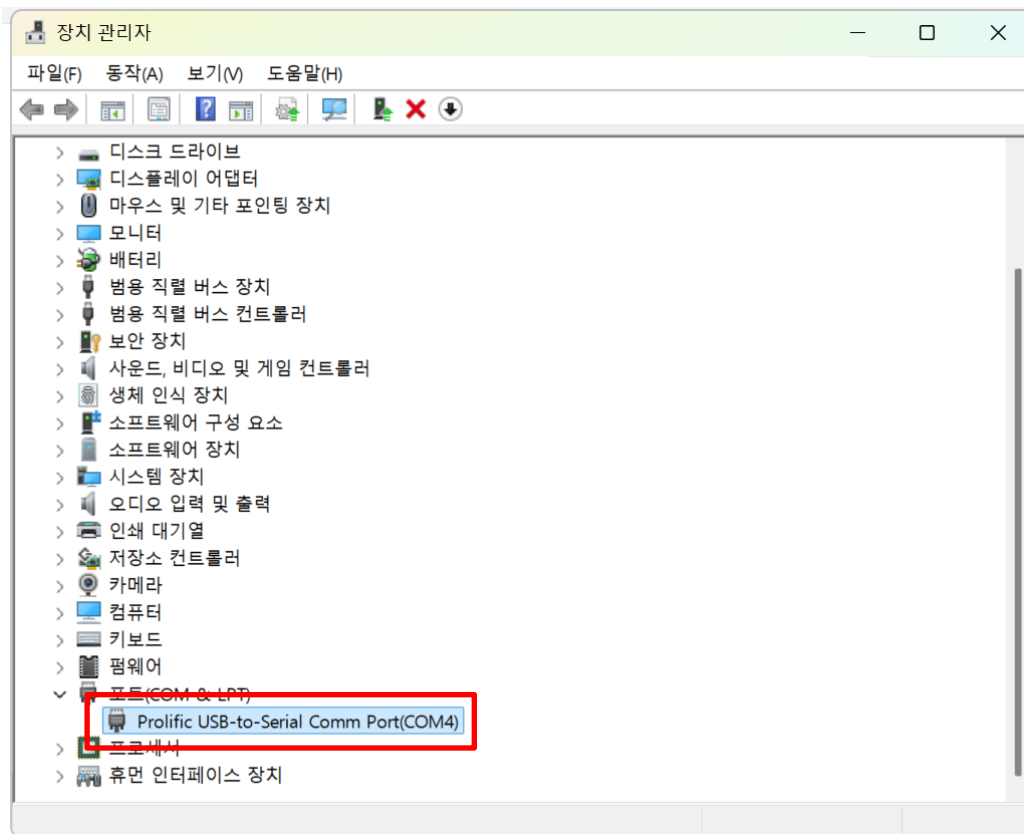
(개인 랩탑) PL2303TA의 경우 드라이버를 구버전으로 바꿔야 Windows 11에서 인식됩니다.

How to connect an FPGA to a terminal via UART



USB-Serial 컨버터를 보드 커넥터에 꽂습니다.

How to connect an FPGA to a terminal via UART

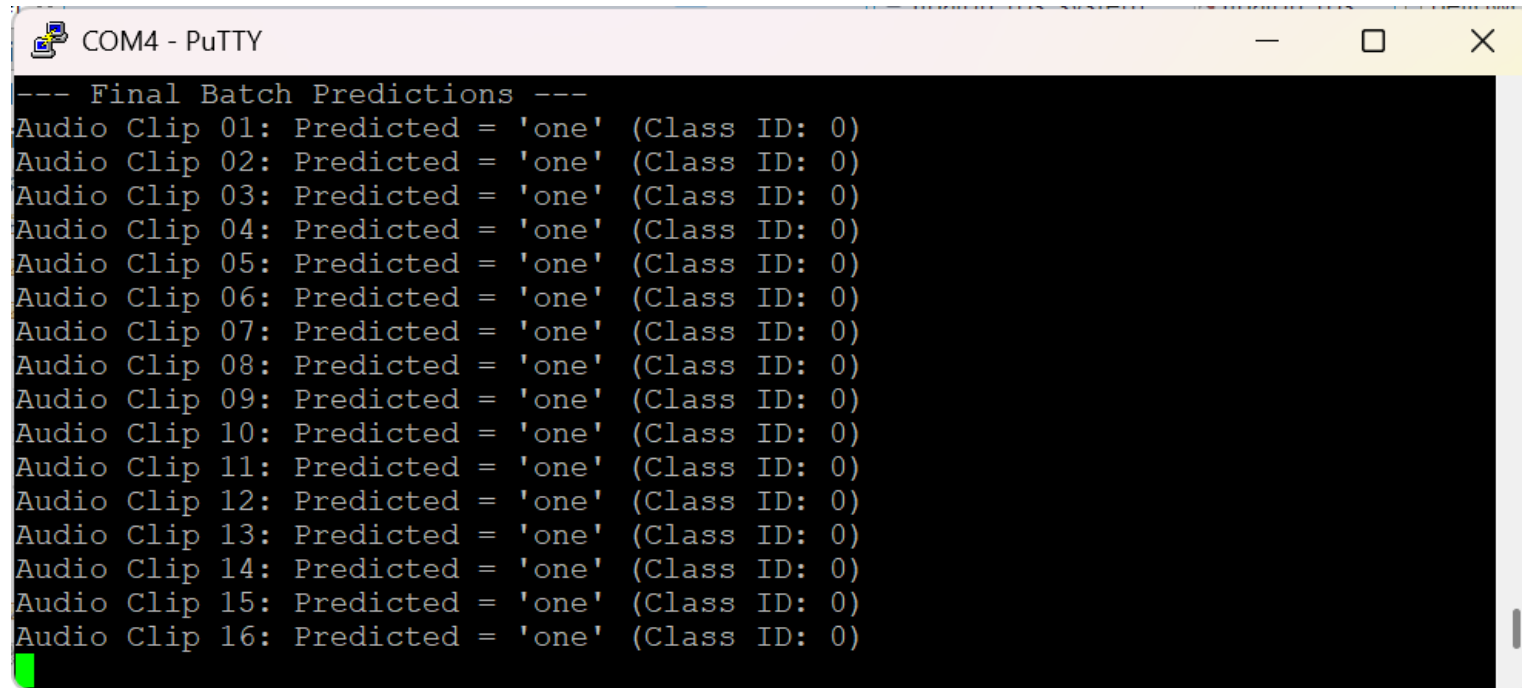


PuTTY를 설치하고 실행합니다.

Connection type: Serial / Serial line: COM{n} / Speed: 115200

으로 설정하고 Open을 누릅니다.

How to connect an FPGA to a terminal via UART

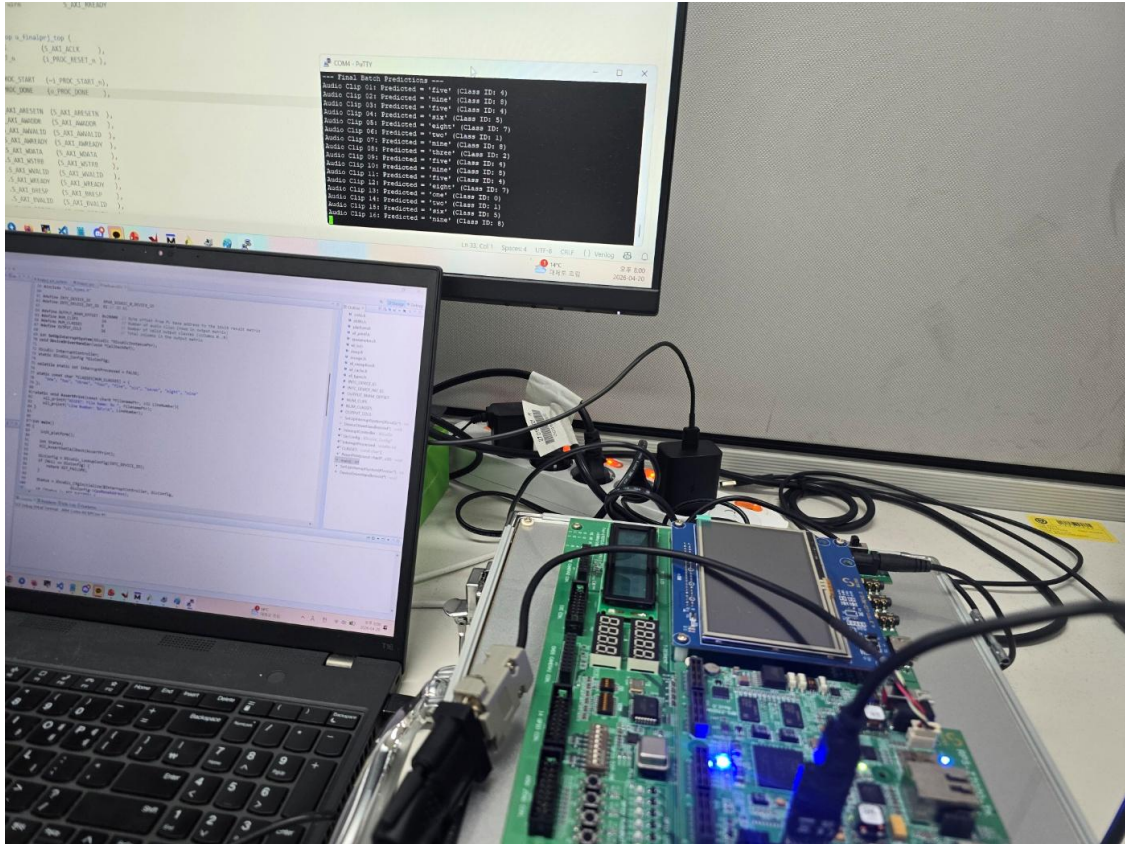


```
COM4 - PuTTY
--- Final Batch Predictions ---
Audio Clip 01: Predicted = 'one' (Class ID: 0)
Audio Clip 02: Predicted = 'one' (Class ID: 0)
Audio Clip 03: Predicted = 'one' (Class ID: 0)
Audio Clip 04: Predicted = 'one' (Class ID: 0)
Audio Clip 05: Predicted = 'one' (Class ID: 0)
Audio Clip 06: Predicted = 'one' (Class ID: 0)
Audio Clip 07: Predicted = 'one' (Class ID: 0)
Audio Clip 08: Predicted = 'one' (Class ID: 0)
Audio Clip 09: Predicted = 'one' (Class ID: 0)
Audio Clip 10: Predicted = 'one' (Class ID: 0)
Audio Clip 11: Predicted = 'one' (Class ID: 0)
Audio Clip 12: Predicted = 'one' (Class ID: 0)
Audio Clip 13: Predicted = 'one' (Class ID: 0)
Audio Clip 14: Predicted = 'one' (Class ID: 0)
Audio Clip 15: Predicted = 'one' (Class ID: 0)
Audio Clip 16: Predicted = 'one' (Class ID: 0)
```

Systolic array 코어를 S1 버튼으로 리셋한 후, S2 버튼을 눌러 출력이 제대로 나오는지 확인합니다.

현재 배포된 코드를 사용한 출력 예시입니다.

How to connect an FPGA to a terminal via UART



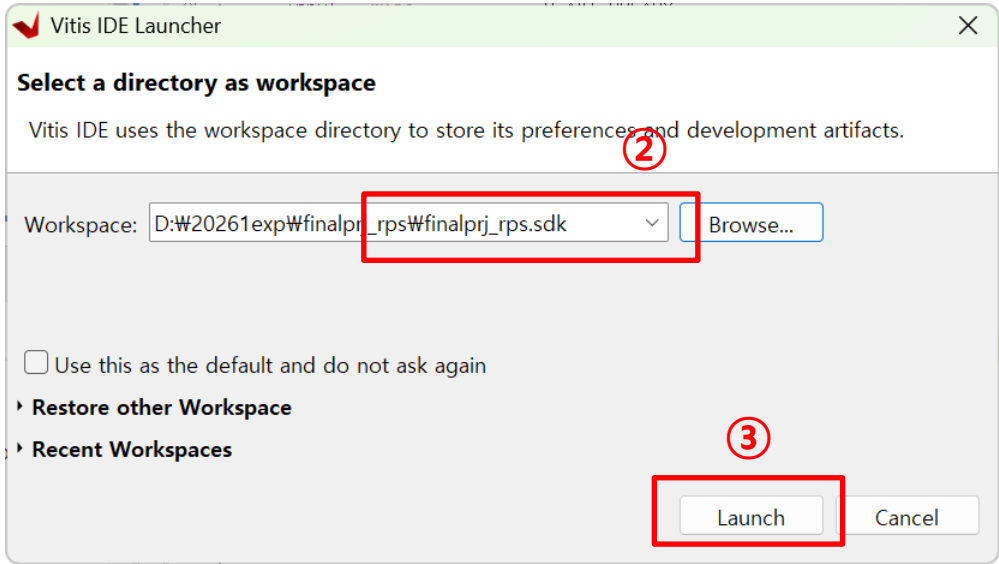
```
COM3 - PuTTY

--- Final Batch Predictions ---
Audio Clip 01: Predicted = 'five' (Class ID: 4)
Audio Clip 02: Predicted = 'nine' (Class ID: 8)
Audio Clip 03: Predicted = 'five' (Class ID: 4)
Audio Clip 04: Predicted = 'six' (Class ID: 5)
Audio Clip 05: Predicted = 'eight' (Class ID: 7)
Audio Clip 06: Predicted = 'two' (Class ID: 1)
Audio Clip 07: Predicted = 'nine' (Class ID: 8)
Audio Clip 08: Predicted = 'three' (Class ID: 2)
Audio Clip 09: Predicted = 'five' (Class ID: 4)
Audio Clip 10: Predicted = 'nine' (Class ID: 8)
Audio Clip 11: Predicted = 'five' (Class ID: 4)
Audio Clip 12: Predicted = 'eight' (Class ID: 7)
Audio Clip 13: Predicted = 'one' (Class ID: 0)
Audio Clip 14: Predicted = 'two' (Class ID: 1)
Audio Clip 15: Predicted = 'six' (Class ID: 5)
Audio Clip 16: Predicted = 'nine' (Class ID: 8)
```

테스트 환경 및 정답 출력 예시

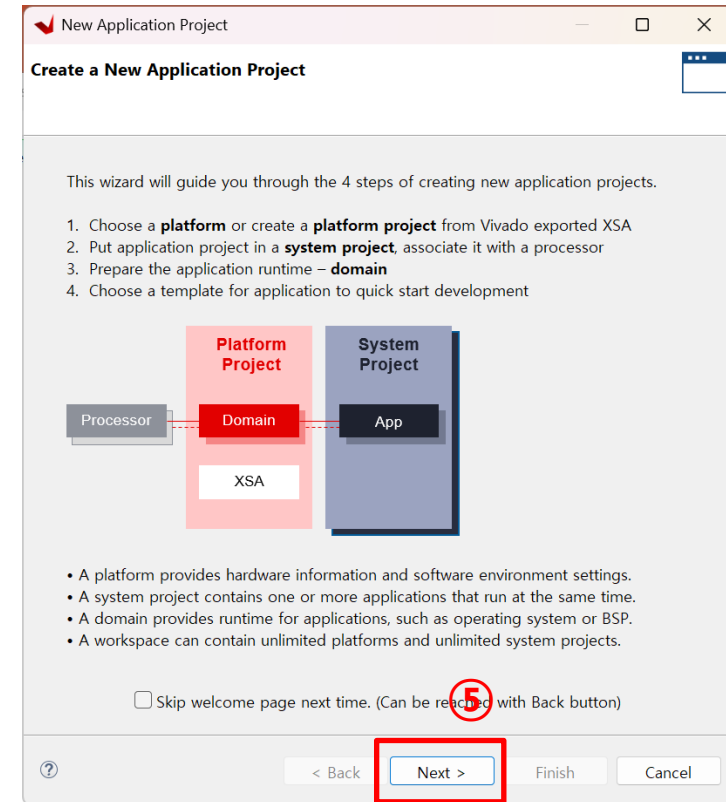
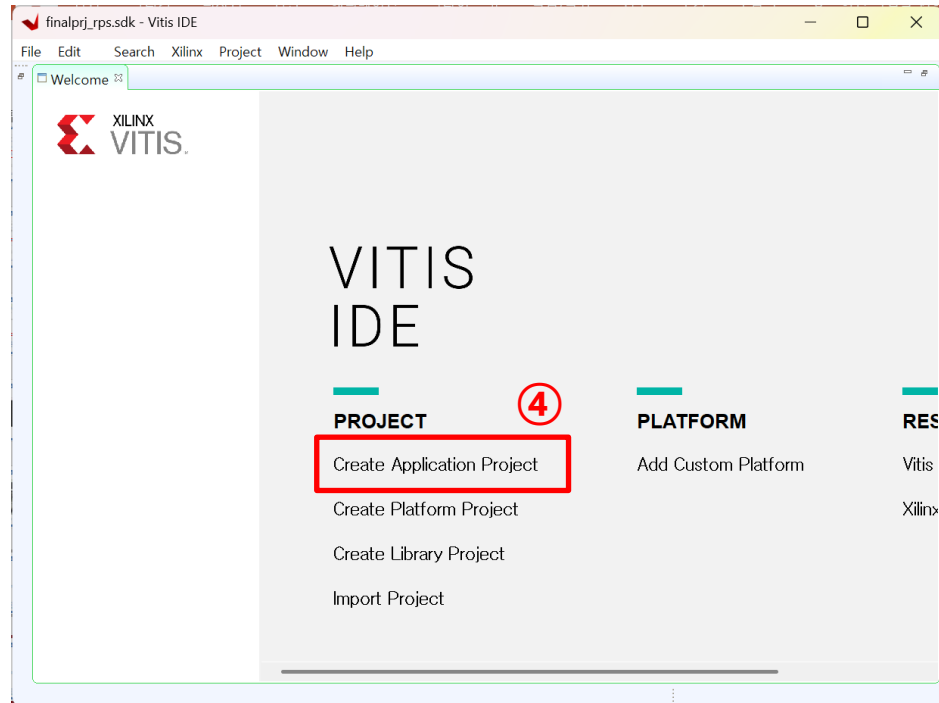
Appendix : Start a new Vitis project

finalprj_rps.xpr	2026-04-20 오후 8:52
rps_z7020_tk.xdc	2026-04-20 오후 6:29
vivado.jou	2026-04-22 오전 10:59
vivado.log	2026-04-22 오전 10:59
vivado_pid45536.str	2026-04-22 오전 11:00
finalprj_rps.sdk	2026-04-22 오전 11:32



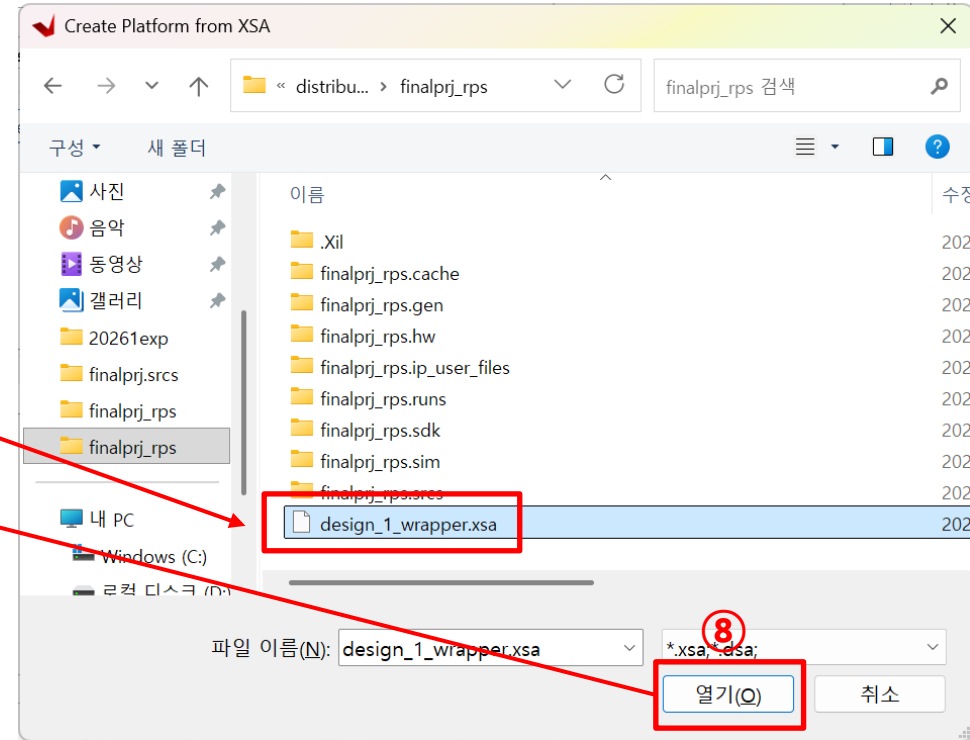
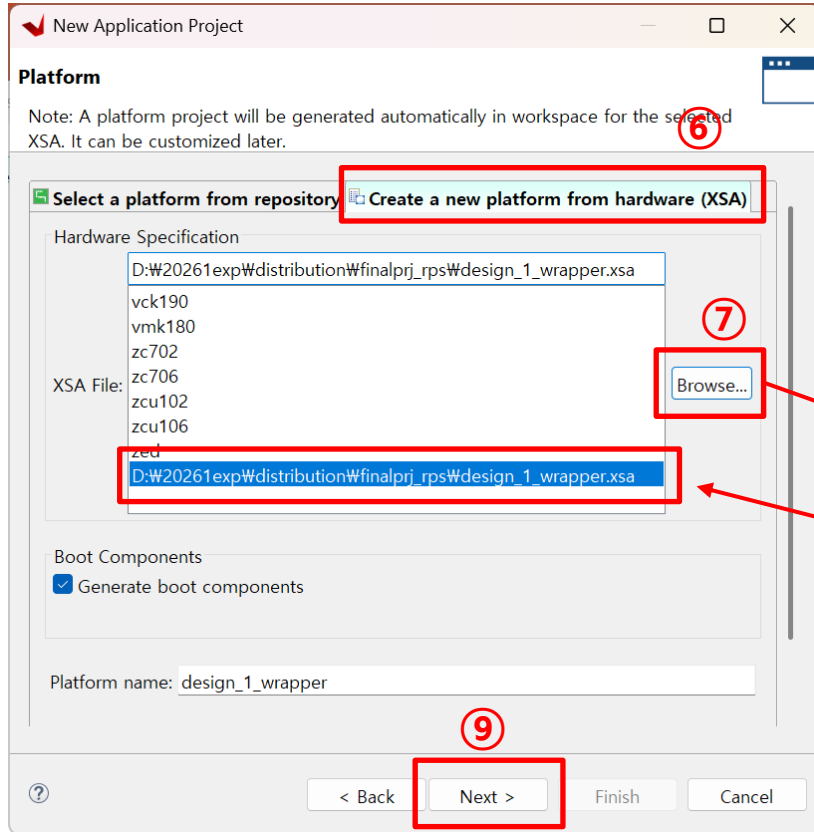
Vitis프로젝트가 완전히 꼬였거나 사용중인 Vivado와 버전이 안 맞을 때 다시 생성할 수도 있습니다.
finalprj_rps 디렉토리 내부 finalprj_rps.sdk 디렉토리를 생성 후,
Vitis프로젝트의 workspace를 해당 디렉토리로 잡고 실행합니다.

Appendix : Start a new Vitis project



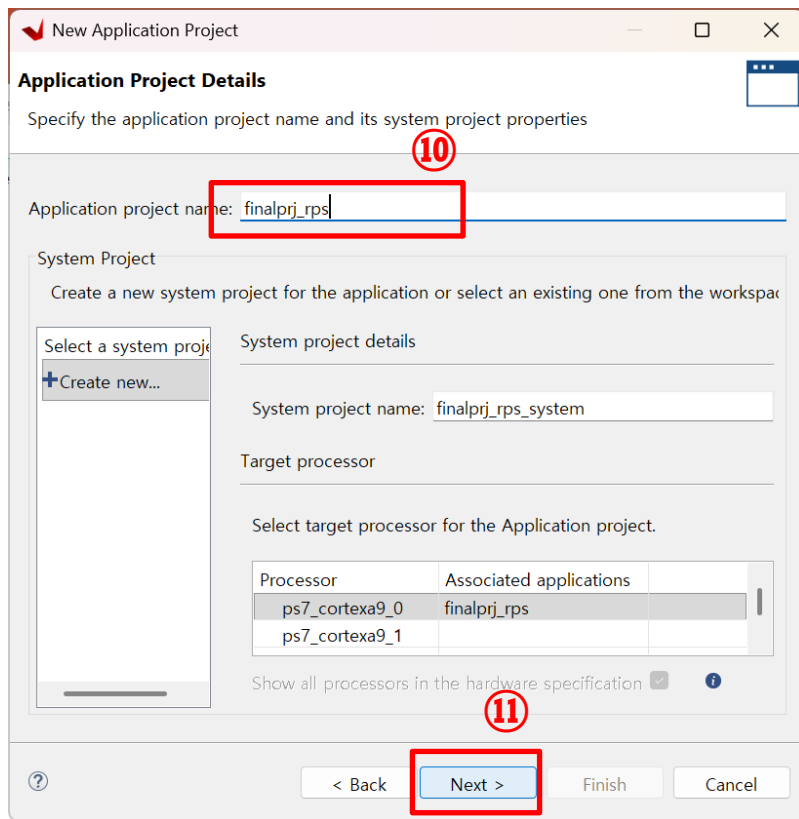
그대로 누르세요

Appendix : Start a new Vitis project



Create a new platform... 탭에서 Browse를 눌러 사용할 .xsa파일을 고르고 열기를 누릅니다.
조금 기다리면 .xsa가 Hardware Specification에 등록되고, 이후 Next를 누릅니다.

Appendix : Start a new Vitis project



New Application Project

Application Project Details

Specify the application project name and its system project properties

Application project name:

System Project

Create a new system project for the application or select an existing one from the workspace

Select a system project

+ Create new...

System project details

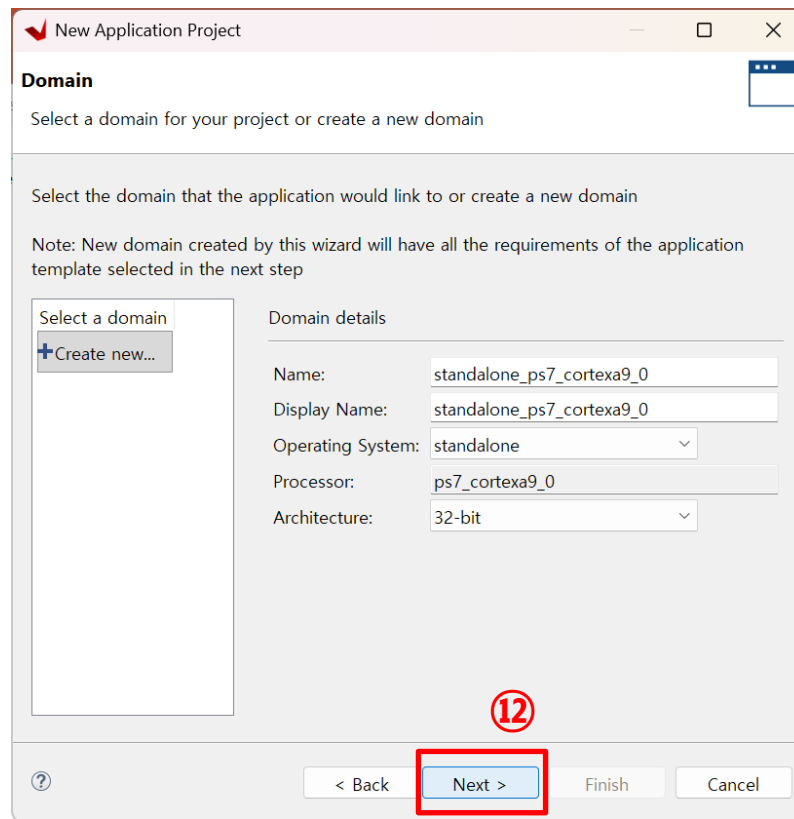
System project name:

Target processor

Select target processor for the Application project.

Processor	Associated applications
ps7_cortexa9_0	finalprj_rps
ps7_cortexa9_1	

Show all processors in the hardware specification ☒ ⓘ



New Application Project

Domain

Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

Select a domain

+ Create new...

Domain details

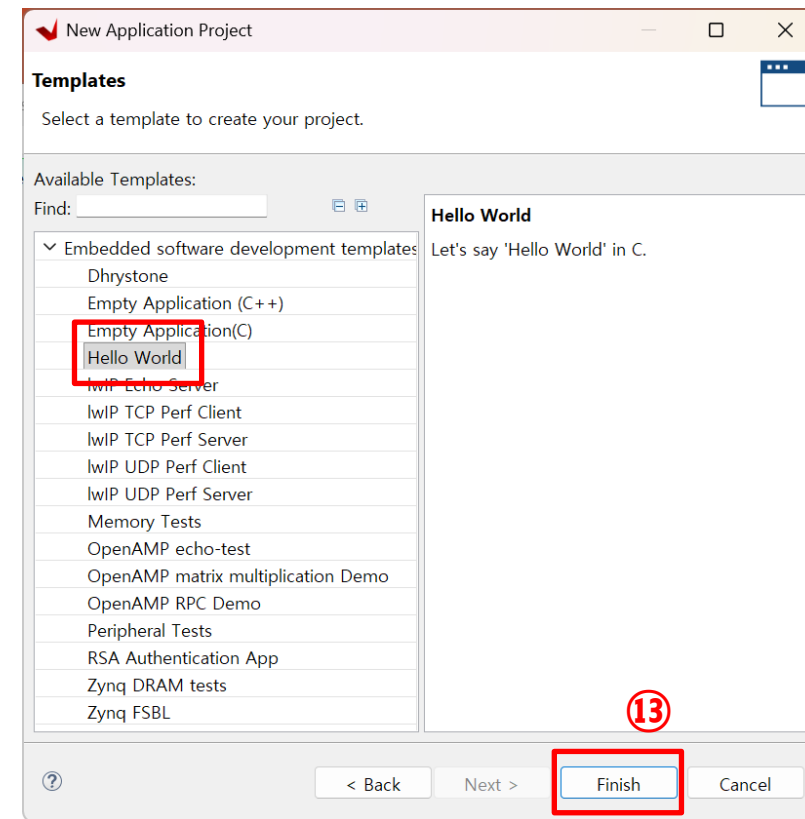
Name:

Display Name:

Operating System:

Processor:

Architecture:



New Application Project

Templates

Select a template to create your project.

Available Templates:

Find:

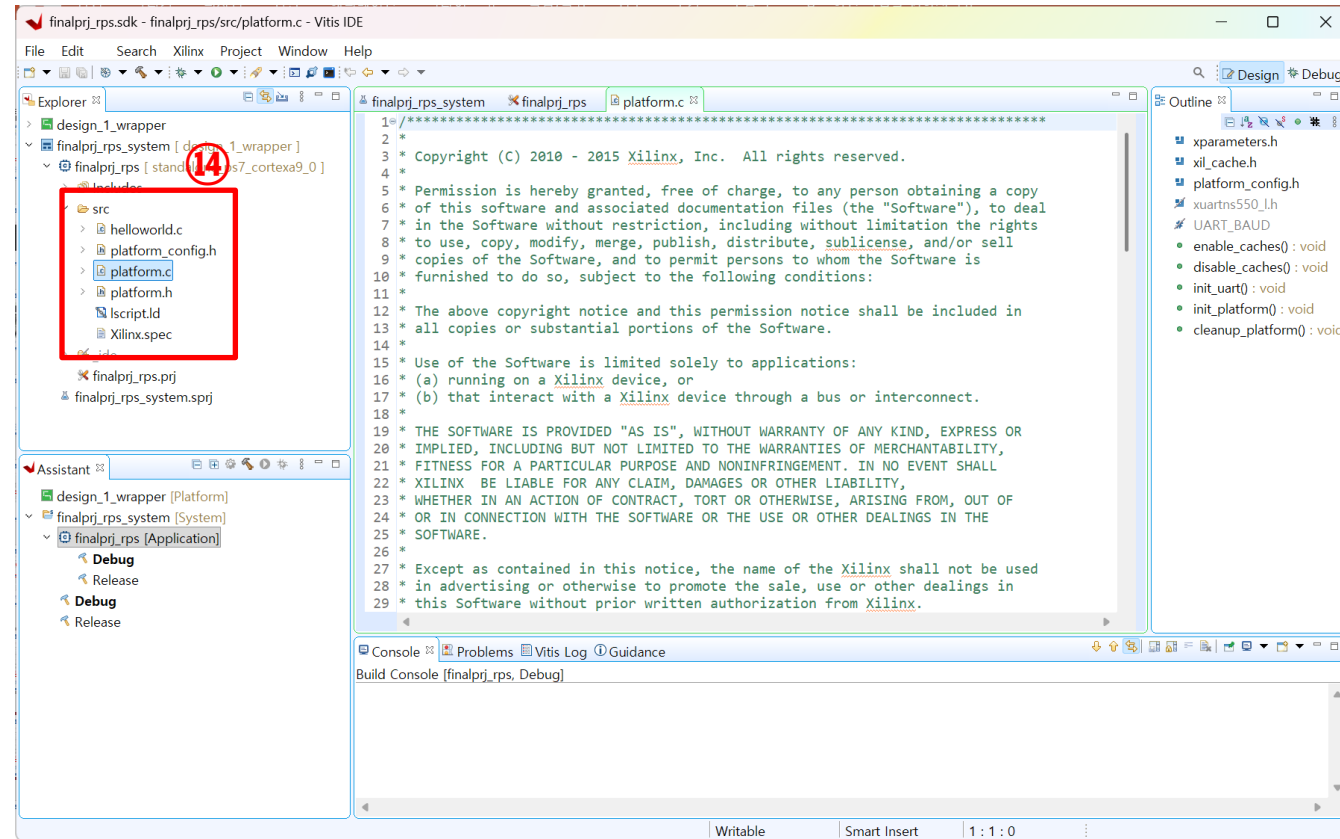
- Embedded software development templates
 - Dhrystone
 - Empty Application (C++)
 - Empty Application (C)
 - Hello World
 - lwIP Echo Server
 - lwIP TCP Perf Client
 - lwIP TCP Perf Server
 - lwIP UDP Perf Client
 - lwIP UDP Perf Server
 - Memory Tests
 - OpenAMP echo-test
 - OpenAMP matrix multiplication Demo
 - OpenAMP RPC Demo
 - Peripheral Tests
 - RSA Authentication App
 - Zynq DRAM tests
 - Zynq FSBL

Hello World

Let's say 'Hello World' in C.

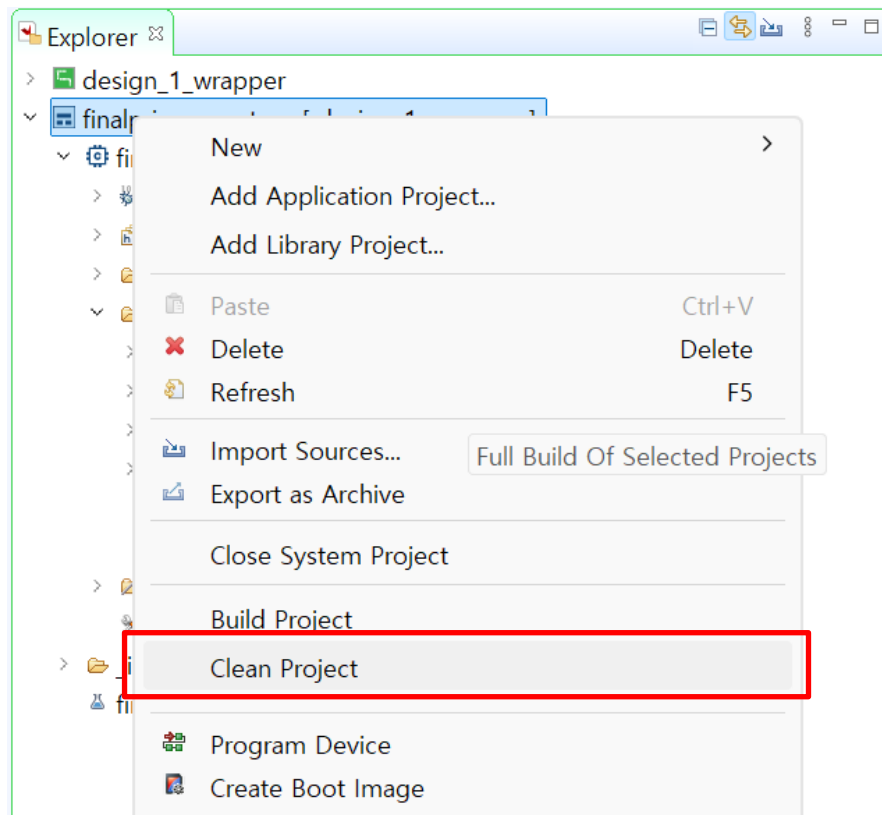
Project name은 finalprj_rps로 짓고 Next를 두 번 누릅니다.
Hello World 프로젝트를 고르고 Finish

Appendix : Start a new Vitis project



src/helloworld.c 파일을 찾아서, finalprj_rps.zip에 있는 helloworld.c파일의 내용으로 대체합니다.
단순히 복붙만 하면 됩니다. 이상으로 새 프로젝트 세팅이 완료됐습니다.

Appendix : Clean a Vitis project



비트스트림 반영이 잘 안 될 경우 Clean Project로 캐시 등을 모두 지울 수 있습니다.
이후 새롭게 빌드가 필요합니다.

Term Project : Systolic array

유의사항 :

1. 시뮬레이션과 보드에서의 동작 차이를 포함하여 term project의 구현에 대한 어떠한 질문도 받지 않습니다. 단, Vivado와 보드와의 연결 및 환경 설정에 대해서는 수업시간에 질문할 수 있습니다.
2. 평가 당일날 새로운 입력 데이터를 배포합니다. 새로운 입력에 대한 추론 결과 또한 프로젝트 평가 요소로 사용됩니다. 평가 당일 Vivado에서 합성을 수행할 수 있는 환경이 갖추어져 있어야 합니다. **노트북 사용 시 어댑터 사용 권장**
3. 평가 당일 정규 수업시간 끝 (13:49) 까지 실제 보드에서의 실행 결과를 확인받고, LearnUs에 보고서 및 전체 프로젝트 압축파일을 제출합니다.
4. **지각 제출은 받지 않으며**, 기한 이후 제출된 프로젝트 파일 및 보고서는 0점 처리됩니다.
5. 기말 프로젝트는 **팀별 프로젝트**이며, **코드 공유, 과도한 AI도구 사용** 등의 부정행위가 적발될 경우 관련 학생은 0점 처리됩니다.

Term Project : Systolic array

보고서 작성 시 필수 서술 사항 :

1. Theory: 시스톨릭 어레이의 작동 원리와 타일링에 대하여 필수 기재합니다.
2. Implementation:
 - [슬라이드 16] 부터 설명한 Structure을 읽고, 상위 수준의 아키텍처 컨셉을 어떻게 실제 하드웨어 맞춰 구현해야 하는지 상세히 서술합니다.
 - [슬라이드 26] 내용을 보고서에 포함해야 합니다.
 - 보고서에 코드 snippet을 많이 넣지 말고, 핵심 코드만을 옮긴 뒤 나머지는 Verilog소스의 주석을 달아야 합니다. (한 snippet은 ½페이지를 넘지 말아야 합니다)
3. Discussion: 성공했다면 어려웠던 점을, 실패했다면 예상되는 실패 사유를 서술합니다.
4. 동작 사진 (캡처+보드와 함께 찍은 사진) 을 반드시 첨부하세요.

Evaluation Criteria

Category	Description	Score
Report	Description of project goal and results discussion	30
RTL	Design block implementation (e.g., MAC, control)	20
Testbench	Functional verification (calculation result)	15
Synthesis	Synthesizable code?	10
Hardware Implementation	Verification on actual FPGA	20
Performance	Speed + design cost	5



Q&A

